

# # Kafka CLI Cheat Sheet (macOS)

Assumptions:

- You are in Kafka home (folder that contains `bin/`).
- Commands use the Kafka shell scripts (`\*.sh`).
- You have a broker reachable on `localhost:9092`.

## ## 0) Quick setup

Set a reusable bootstrap variable:

```
```bash
export BS=localhost:9092
# Multi-broker example:
# export BS=localhost:9092,localhost:9094
```
```

---

## ## 1) Start/Stop Kafka (KRaft)

Start brokers (example):

```
```bash
./bin/kafka-server-start.sh config/server-1.properties
./bin/kafka-server-start.sh config/server-2.properties
./bin/kafka-server-start.sh config/server-3.properties
```
```

Stop brokers (best-effort):

```
```bash
./bin/kafka-server-stop.sh
```
```

---

## ## 2) Topics

### ### List topics

```
``bash
./bin/kafka-topics.sh --bootstrap-server $BS --list
``
```

### ### Describe a topic (partitions, leader, replicas)

```
``bash
./bin/kafka-topics.sh --bootstrap-server $BS --describe --topic product-created-events-
topic
``
```

### ### Create a topic

Example: 3 partitions, replication factor 1

```
``bash
./bin/kafka-topics.sh --bootstrap-server $BS \
--create --topic product-created-events-topic \
--partitions 3 --replication-factor 1
``
```

### ### Create a DLT topic (match partitions)

If your main topic has 3 partitions, create the DLT with 3 partitions too:

```
``bash
./bin/kafka-topics.sh --bootstrap-server $BS \
--create --topic product-created-events-topic-dlt \
--partitions 3 --replication-factor 1
``
```

### ### Alter partitions (only increase)

```
``bash
```

```
./bin/kafka-topics.sh --bootstrap-server $BS \  
--alter --topic product-created-events-topic-dlt --partitions 3  
```\n
```

### ### Delete a topic

```
```\nbash  
./bin/kafka-topics.sh --bootstrap-server $BS --delete --topic product-created-events-topic  
```\n
```

Notes:

- Topic deletion can be controlled by broker config `delete.topic.enable=true`.

---

## ## 3) Produce messages

### ### Produce values

```
```\nbash  
./bin/kafka-console-producer.sh --bootstrap-server $BS --topic product-created-events-  
topic  
```\n
```

### ### Produce with keys (key:value)

```
```\nbash  
./bin/kafka-console-producer.sh --bootstrap-server $BS \  
--topic product-created-events-topic \  
--property parse.key=true \  
--property key.separator=:  
```\n
```

Example input:

```
```\ntext  
user1:{"productId":"1","title":"iPhone","price":1500,"quantity":1}  
```\n
```

---

## **## 4) Consume messages**

### **### Consume from latest**

```
```bash
./bin/kafka-console-consumer.sh --bootstrap-server $BS --topic product-created-events-
topic
```
```

### **### Consume from beginning**

```
```bash
./bin/kafka-console-consumer.sh --bootstrap-server $BS \
--topic product-created-events-topic --from-beginning
```
```

### **### Print key/partition/offset**

```
```bash
./bin/kafka-console-consumer.sh --bootstrap-server $BS \
--topic product-created-events-topic --from-beginning \
--property print.key=true \
--property print.partition=true \
--property print.offset=true \
--property print.value=true
```
```

### **### Consume a DLT**

```
```bash
./bin/kafka-console-consumer.sh --bootstrap-server $BS \
--topic product-created-events-topic-dlt --from-beginning \
--property print.key=true --property print.value=true
```
```

---

## **## 5) Consumer groups**

### ### List groups

```
```bash
./bin/kafka-consumer-groups.sh --bootstrap-server $BS --list
```
```

### ### Describe a group (lag, offsets)

```
```bash
./bin/kafka-consumer-groups.sh --bootstrap-server $BS \
--describe --group product-created-events
```
```

### ### Reset offsets (reprocess)

Dry run:

```
```bash
./bin/kafka-consumer-groups.sh --bootstrap-server $BS \
--group product-created-events \
--topic product-created-events-topic \
--reset-offsets --to-earliest --dry-run
```
```

Execute:

```
```bash
./bin/kafka-consumer-groups.sh --bootstrap-server $BS \
--group product-created-events \
--topic product-created-events-topic \
--reset-offsets --to-earliest --execute
```
```

---

## ## 6) Topic configuration (retention, etc.)

### ### Describe topic configs

```
```bash
```

```
./bin/kafka-configs.sh --bootstrap-server $BS \  
--entity-type topics --entity-name product-created-events-topic \  
--describe  
```\
```

### ### Set retention (example: 1 hour)

```
```\bash  
./bin/kafka-configs.sh --bootstrap-server $BS \  
--entity-type topics --entity-name product-created-events-topic \  
--alter --add-config retention.ms=3600000  
```\
```

### ### Remove an override

```
```\bash  
./bin/kafka-configs.sh --bootstrap-server $BS \  
--entity-type topics --entity-name product-created-events-topic \  
--alter --delete-config retention.ms  
```\
```

---

## ## 7) Useful debug commands

### ### Check broker API versions

```
```\bash  
./bin/kafka-broker-api-versions.sh --bootstrap-server $BS  
```\
```

### ### Check KRaft quorum status

```
```\bash  
./bin/kafka-metadata-quorum.sh --bootstrap-server $BS describe --status  
```\
```

---

## ## 8) Common scenarios

### ### Scenario A: Create main topic + DLT correctly

```
```bash
./bin/kafka-topics.sh --bootstrap-server $BS --create \
--topic product-created-events-topic --partitions 3 --replication-factor 1

./bin/kafka-topics.sh --bootstrap-server $BS --create \
--topic product-created-events-topic-dlt --partitions 3 --replication-factor 1
```
```

### ### Scenario B: DLT exists but has fewer partitions

```
```bash
./bin/kafka-topics.sh --bootstrap-server $BS \
--alter --topic product-created-events-topic-dlt --partitions 3
```
```

### ### Scenario C: Delete unwanted topics

```
```bash
./bin/kafka-topics.sh --bootstrap-server $BS --delete --topic product-created-events-topic-
dlt-2
./bin/kafka-topics.sh --bootstrap-server $BS --delete --topic product-created-events-topic-
dlt-3
```
```

## Kafka Command Reference?

---

## **Section 2: Apache Kafka Broker**

# Kafka (KRaft Mode) Setup & Commands – Formatted Documentation

## 1. Generate Cluster UUID

```
bin/kafka-storage.sh random-uuid
```

Example output:

```
an-adF1MQXebx1sNK7l2Mw
```

### ◇ What it does:

Generate a **unique Cluster ID** for your Kafka cluster.

- Kafka needs a permanent ID to identify a cluster.
- This ID is written in Kafka's metadata storage.
- Acts like a *fingerprint* for your Kafka installation.
- Generate once and reuse all brokers in the same cluster.

## 2. Format Kafka Storage (Single Node)

```
bin/kafka-storage.sh format \  
--config config/server.properties \  
--cluster-id i5obklXcSdKLvoyyYnp6-Q \  
--standalone
```

### ◇ What it does:

Initializes Kafka's metadata storage in **KRaft mode**.

This command:

- Creates internal Kafka directories
- Stores the cluster ID
- Initializes controller metadata
- Prepares Kafka to run without ZooKeeper
- Marks this node as both **Broker + Controller**

### Options Explained:

- `--config config/server.properties`  
→ Uses the Kafka configuration file
- `--cluster-id ...`  
→ Permanently attaches this node to the Kafka cluster
- `--standalone`  
→ Single-node setup (broker + controller together)

⚠ Important:

- Run **only once**
- Running again deletes Kafka metadata
- Equivalent to *formatting a hard disk before installing an OS*

## 3. Start Kafka Server

`bin/kafka-server-start.sh config/server.properties`

### ♦ What it does:

Starts the Kafka server process.

It:

- Launches Kafka in KRaft mode
- Starts:
  - **Broker** → handles messages
  - **Controller** → manages metadata, partitions, leader election

- Opens ports:
  - 9092 → Client communication
  - 9093 → Controller communication

Once started:

- Kafka is live
- Producers/Consumers can connect
- Spring Boot apps can publish & consume events

## Running Multiple Kafka Brokers (3-Node Cluster)

### Step 1: Create Configuration Files

Copy server.properties three times:

```
config/server-1.properties  
config/server-2.properties  
config/server-3.properties
```

### Step 2: Modify Each Server Configuration

#### ◇ server-1.properties

```
node.id=1
```

```
# Broker & Controller ports  
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
```

controller.quorum.bootstrap.servers=localhost:9093,localhost:9095,localhost:9097

advertised.listeners=PLAINTEXT://localhost:9092,CONTROLLER://localhost:9093

log.dirs=/tmp/server-1/kraft-combined-logs

### ◇ server-2.properties

```
node.id=2
```

```
listeners=PLAINTEXT://:9094,CONTROLLER://:9095
```

```
controller.quorum.bootstrap.servers=localhost:9093,localhost:9095,  
localhost:9097
```

```
advertised.listeners=PLAINTEXT://localhost:9094,CONTROLLER://loca  
lhost:9095
```

```
log.dirs=/tmp/server-2/kraft-combined-logs
```

### ◇ server-3.properties

```
node.id=3
```

```
listeners=PLAINTEXT://:9096,CONTROLLER://:9097
```

```
controller.quorum.bootstrap.servers=localhost:9093,localhost:9095,  
localhost:9097
```

```
advertised.listeners=PLAINTEXT://localhost:9096,CONTROLLER://loca  
lhost:9097
```

```
log.dirs=/tmp/server-3/kraft-combined-logs
```

## Step 3: Format Storage for Each Broker

Use the **same Cluster ID** for all brokers.

### a) Format Server 1

```
./bin/kafka-storage.sh format \  
-t an-adF1MQXebx1sNK7l2Mw \  
-c config/server-1.properties \  
--standalone
```

Output:

Formatting dynamic metadata voter directory /tmp/server-1/kraft-combined-logs with metadata.version 4.1-IV1.

### b) Format Server 2

```
./bin/kafka-storage.sh format \  
-t an-adF1MQXebx1sNK7l2Mw \  
-c config/server-2.properties \  
--standalone
```

Output:

Formatting dynamic metadata voter directory /tmp/server-2/kraft-combined-logs with metadata.version 4.1-IV1.

### c) Format Server 3

```
./bin/kafka-storage.sh format \  
-t an-adF1MQXebx1sNK7I2Mw \  
-c config/server-3.properties \  
--standalone
```

Output:

Formatting dynamic metadata voter directory /tmp/server-3/kraft-combined-logs with metadata.version 4.1-IV1.

⚠ Run these **only once per broker**.

## Step 5: Start All Kafka Servers

Open 3 terminals:

```
./bin/kafka-server-start.sh config/server-1.properties
```

```
./bin/kafka-server-start.sh config/server-2.properties
```

```
./bin/kafka-server-start.sh config/server-3.properties
```

Now you have:

- 3 Kafka brokers
- 3 Controllers
- Fully working Kafka KRaft cluster

## Step 7: Stop Kafka Servers

```
./bin/kafka-server-stop.sh
```

(Repeat in all terminals or kill each process)

## Interview Ready Summary

In Kafka 4.x, ZooKeeper is removed and Kafka runs in KRaft mode.

We first generate a cluster UUID, then format Kafka storage to initialize metadata and controller state.

Each broker must be formatted once with the same cluster ID.

Finally, we start each Kafka server using its own configuration file to form a multi-node Kafka cluster.

## **Section 3: Apache Kafka CLI: Topics**

## Creating a new Kafka Topic?

|                               |
|-------------------------------|
| Step 1: Generate Cluster UUID |
|-------------------------------|

|                                               |
|-----------------------------------------------|
| <pre>./bin/kafka-storage.sh random-uuid</pre> |
|-----------------------------------------------|

|                                               |
|-----------------------------------------------|
| Step 2: Format Kafka Storage for all 3 server |
|-----------------------------------------------|

|                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------|
| <pre>./bin/kafka-storage.sh format --config config/server-1.properties --cluster-id an-adF1MQXebx1sNK7I2Mw --standalone</pre> |
|-------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------|
| <pre>./bin/kafka-storage.sh format --config config/server-2.properties --cluster-id an-adF1MQXebx1sNK7I2Mw --standalone</pre> |
|-------------------------------------------------------------------------------------------------------------------------------|

|                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------|
| <pre>./bin/kafka-storage.sh format --config config/server-3.properties --cluster-id an-adF1MQXebx1sNK7I2Mw --standalone</pre> |
|-------------------------------------------------------------------------------------------------------------------------------|

|                                    |
|------------------------------------|
| Step 3: Start all 3 Kafka servers. |
|------------------------------------|

|                                                                   |
|-------------------------------------------------------------------|
| <pre>./bin/kafka-server-start.sh config/server-1.properties</pre> |
|-------------------------------------------------------------------|

|                                                                   |
|-------------------------------------------------------------------|
| <pre>./bin/kafka-server-start.sh config/server-2.properties</pre> |
|-------------------------------------------------------------------|

|                                                                   |
|-------------------------------------------------------------------|
| <pre>./bin/kafka-server-start.sh config/server-3.properties</pre> |
|-------------------------------------------------------------------|

Partitions can be no. Of consumer.

Replication parameter specifies how many copies of each partition are stored across different brokers or different servers in my Kafka cluster.

And this is a way to achieve reliability or fault tolerance in Kafka Cluster because it allows data to be available even if some brokers fail or become unreachable.

For example, if you set replication factor to three, then each partition will have three replicas, one of which will be leader and other two will be followers.

But the number you specify here cannot be greater than the number of brokers that you have in your cluster.

So if you start at one server only, then you cannot have replication factor greater than one.

But if you start three servers in a cluster, then you can specify replication factor three, and each broker will have a copy of a partition.

|  |
|--|
|  |
|  |

## Section 6: Kafka Producer – Spring Boot Microservice

## **Introduction to Kafka Producer?**

Apache Kafka brokers accept messages or events from Kafka producers.

Kafka producer is a spring boot microservice that publishes events to Kafka Broker.

So the primary role of Kafka Producer is to publish messages or events to one or more Kafka topics.

In a Spring boot application, this is typically achieved using spring for Apache Kafka library, which simplifies the process of integrating Kafka functionality into your spring application.

Before sending a message, Kafka producer serializes it into a byte array.

So another responsibility of Kafka Producer is to serialize messages to binary format that can be transmitted over the network to Kafka brokers.

Another responsibility of Kafka producer is to specify the name of Kafka topic, where these messages should be sent.

In this particular use case, I call Kafka topic, the product created event topic, and it will store product created events.

When sending a new message to Kafka topic, we will also specify the partition where we want to store it, although this parameter is optional and if we do not specify topic partition, then Kafka Producer will make this decision for us.

And if neither partition nor key is provided, then Kafka producer will distribute messages across partitions in a round robin fashion, ensuring a balanced load.

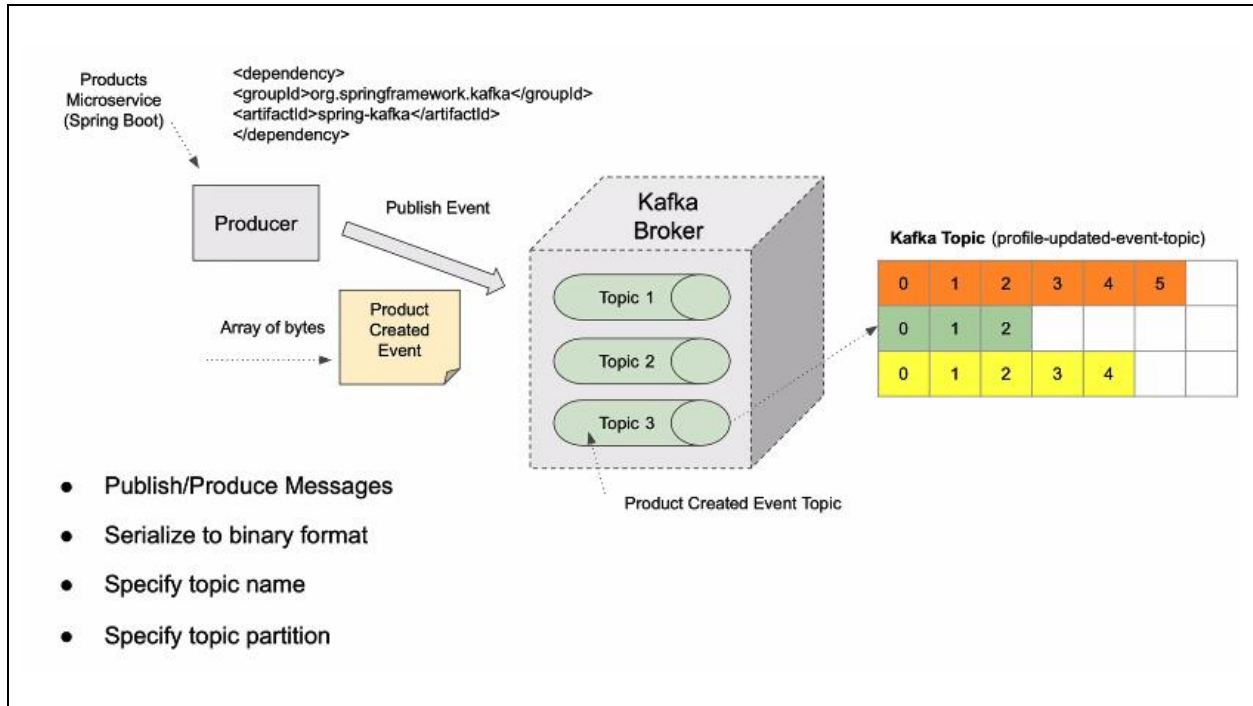
If a partition is not specified but a message key is provided, then Kafka will hash the key and use the result to determine the partition, and this ensures that all messages with the same key go to the same partition.

So another responsibility of Kafka producer is to choose which partition to write event data to.

## **Responsibility of Kafka Producer:**

- Publish/Produce Messages
- Serialize to binary format

- Specify topic name
- Specify topic partitions



## Kafka Producer – Introduction to synchronous communication style?

Let's assume that we have a mobile application that sends a Http request to spring boot microservice.

And let's assume that this spring boot microservice is called orders microservice.

This microservice application, it exposes API endpoints that allow to place new order, view existing orders, or maybe cancel or track order status as well.

So a mobile application can send a Http request to create a new order, for example.

And in this particular case, the mobile application will most likely use synchronous communication style with orders microservice and it will wait for a response.

It will wait for a response because it does want to know for sure if order was successfully created or not.

So synchronous communication style is when sender sends a request and then waits for a response before proceeding.

In the following lectures we will create and we will configure Spring Boot microservice to work as Kafka producer.

And we will use a special class from Spring Framework that is called Kafka Template.

This class encapsulates Kafka Producer and provides a very user friendly way to interact with Kafka from Spring Boot application.

So our orders microservice will create a new object.

It will be called Order Created Event, and it will use Kafka Producer to send a message to Kafka Broker.

If orders microservice wants to know that the order created event was successfully stored in Kafka Broker,

it will use Kafka Producer to send Kafka message using synchronous communication style, which means that it will wait for acknowledgement from Kafka Broker to confirm that the message was successfully received and stored before it proceeds with sending next message or performing other operations.

Because Kafka Producer is waiting to receive a response, it is blocked or it is on hold until the response from Kafka Broker is received.

Once Kafka Broker successfully stores Kafka message in a topic, it will send the response to Kafka Producer that everything is okay and that the message has been stored.

And once Kafka producer receives this acknowledgement that the message is stored successfully, or this microservice will respond back to mobile application with Http response.

So in this diagram, I used synchronous communication style between mobile application and spring boot microservice, and between Kafka Producer and Kafka Broker.

Using synchronous communication style between Kafka Producer and Kafka broker is considered to be more reliable because it ensures that message was successfully stored before moving on to the next task.

But because producer is waiting, it can make your system just a little bit slower.

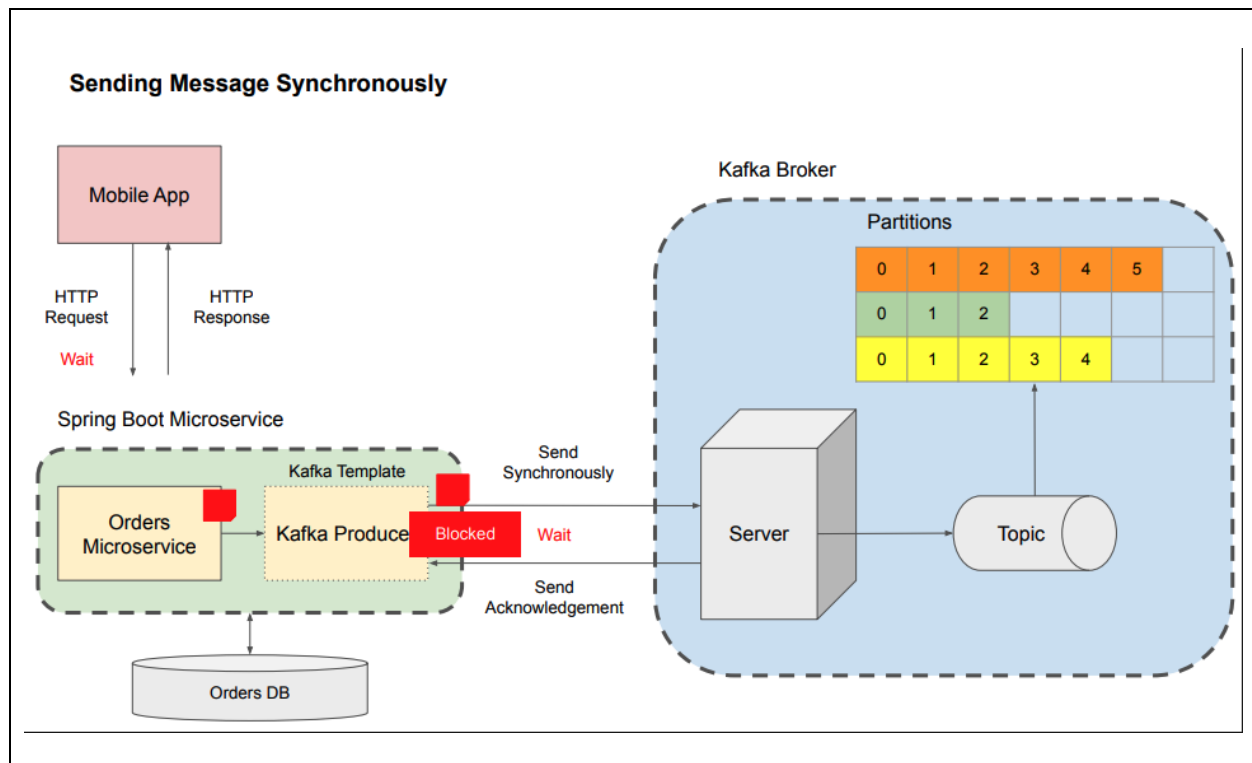
In those cases where high throughput and low latency is required, you might want to consider a different communication style that is called asynchronous communication.

When using a synchronous communication style, Kafka producer sends a message to Kafka Broker and synchronously, but it is not put on hold after and it is not blocked.

While waiting for response from Kafka Broker, Kafka producer sends a message to Kafka Broker and it continues its execution without being blocked or put on hold.

It can still receive a response from Kafka Broker, but it will receive it at a later point.

And that is the main difference between synchronous and asynchronous communication styles.



## Kafka Producer – A use case for asynchronous communication style?

Let's assume that we have a mobile application that displays user login page.

Before user can proceed, they will need to enter their username and password to authenticate.

And once user enters their username and password and taps on login button, mobile application will send a Http request to a login API endpoint.

In this case, mobile application will use synchronous communication style and will wait for Spring boot microservice to respond.

Let's assume that user authentication is handled by authentication microservice.

Authentication Microservice will check its database to see if user with provided username and password exists, and if authentication is successful, it will create a new **UserLoggedEvent** event.

It will then use Kafka Producer to publish this event to Kafka Broker.

Now the purpose of this **UserLoggedEvent** event is only informational and it is needed for analytics purposes only.

Even if this event fails and does not get stored in Kafka topic, we don't want our user to wait or receive error message.

So in this case, we make Kafka producer send message asynchronously.

If we send message asynchronously, Kafka producer will not be blocked, it will not be put on hold and will continue its execution right away.

And because user authentication is successful, our microservice will return a successful Http response.

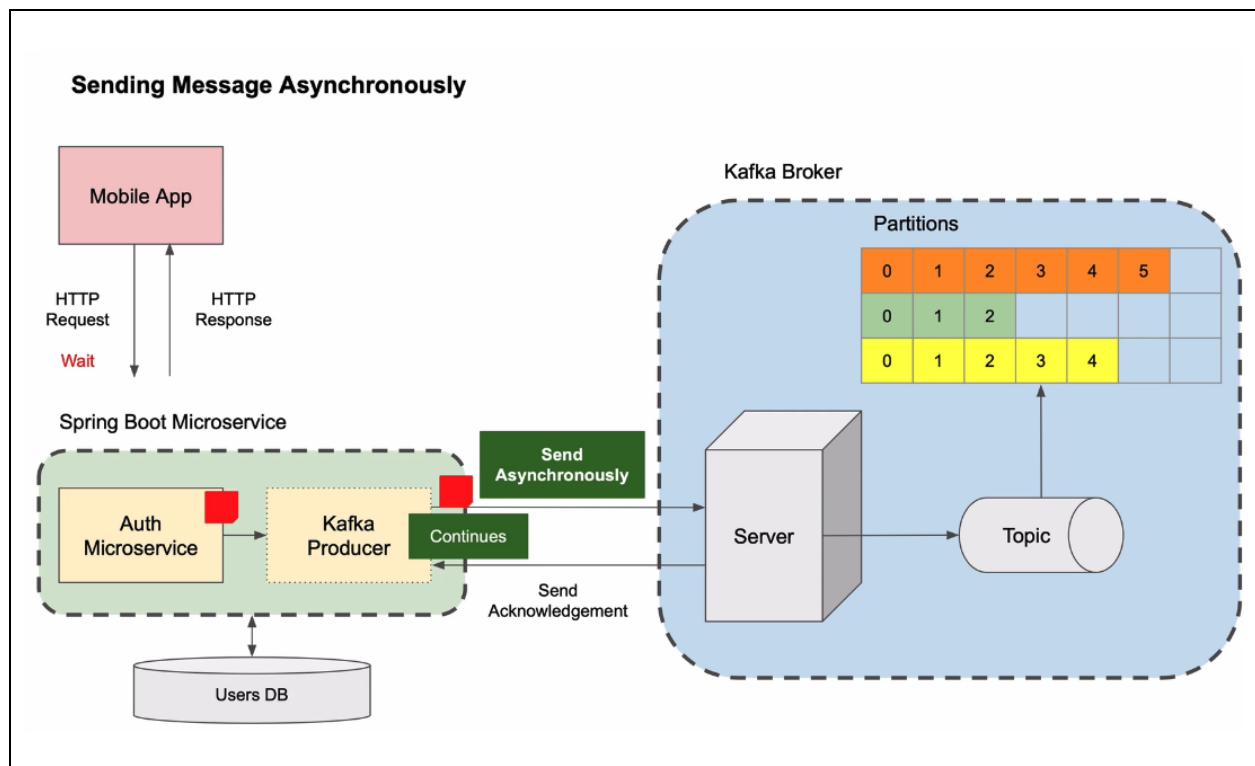
So the user of mobile application will log in and will start using it without waiting for Kafka Broker to acknowledge that it has received an event.

Meanwhile, because **UserLoggedEvent** event was published, Kafka Broker will receive it and it will store it in Kafka topic.

But even if this operation fails and Kafka Broker does not store this event successfully, it will not affect user of our mobile application.

Now, even though Kafka producer does send message asynchronously if needed, we can still make our microservice receive and process response from Kafka Broker.

Once Kafka Broker sends its acknowledgment, we can handle it and process it with asynchronous callback function.



## Section 7: Kafka Producer: Acknowledgement and Retries



## Kafka Producer Acknowledgement?

When a Kafka **producer** sends a record, it is sent to a **Kafka broker**. The broker writes the record to a **topic**, and within that topic the record is stored in a specific **partition**.

### Replication, Leader, and Followers

If your Kafka cluster uses a **replication factor > 1**, each partition is replicated across multiple brokers:

- One broker is elected as the **leader** for that partition.
- Other brokers host replicas as **followers** (replicas).
- The producer writes to the **leader**. The leader then replicates the record to follower replicas.

### Why acknowledgements matter

After the leader receives and writes the record, Kafka sends an **acknowledgement (ack)** back to the producer based on the producer's configuration.

If the leader **acknowledges too early** (before followers replicate) and then crashes permanently, the record may be **lost**. To improve durability, you can configure the producer to wait for acknowledgements from more replicas.

### Producer acks settings (durability vs latency)

#### acks=1 (Leader only)

- The producer waits for an acknowledgement from the **leader broker only**.
- **Pros:** good latency / throughput
- **Cons:** record may be lost if the leader fails before followers replicate

This is commonly the default and is a practical balance between speed and safety.

#### acks=all (All in-sync replicas)

- The producer waits for acknowledgements from **all in-sync replicas (ISR)**.
- **Pros:** strongest durability; data is not lost as long as at least one ISR remains alive
- **Cons:** higher latency; slower writes because the producer waits for multiple brokers

### Spring Kafka example property:

- `spring.kafka.producer.acks=all`

#### acks=0 (No acknowledgements)

- The producer does **not** wait for any acknowledgement.
- **Pros:** fastest possible producer performance
- **Cons:** no guarantee the broker received or stored the record (possible silent loss)

**Typical use case:** high-volume, low-criticality data like frequent GPS location updates where occasional loss is acceptable.

**Does acks=all get slower as you add more brokers?**

**Yes and no.**

- **No**, because the producer does **not** wait for every broker in the cluster.
- It waits only for the **in-sync replicas (ISR)** for the partition being written to.

The number of replicas involved depends on the topic's **replication factor**, which is set when the topic is created.

**Controlling how many replicas must acknowledge:** min.insync.replicas

Kafka also lets you configure a minimum number of replicas that must be in-sync to accept writes durably:

- min.insync.replicas=2 (example)

This means:

- Even if a partition has (say) 5 replicas, Kafka can be configured so that only **2 in-sync replicas** must successfully store the record for the write to be considered valid (when using acks=all).

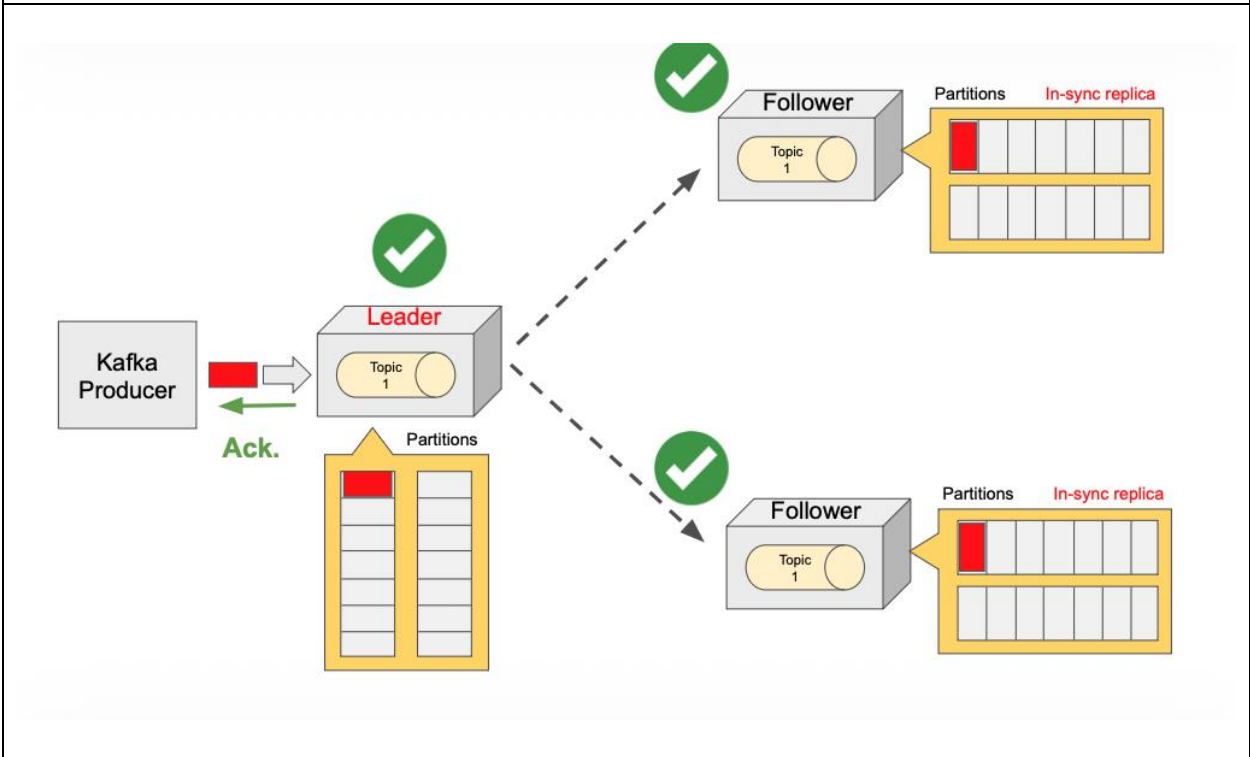
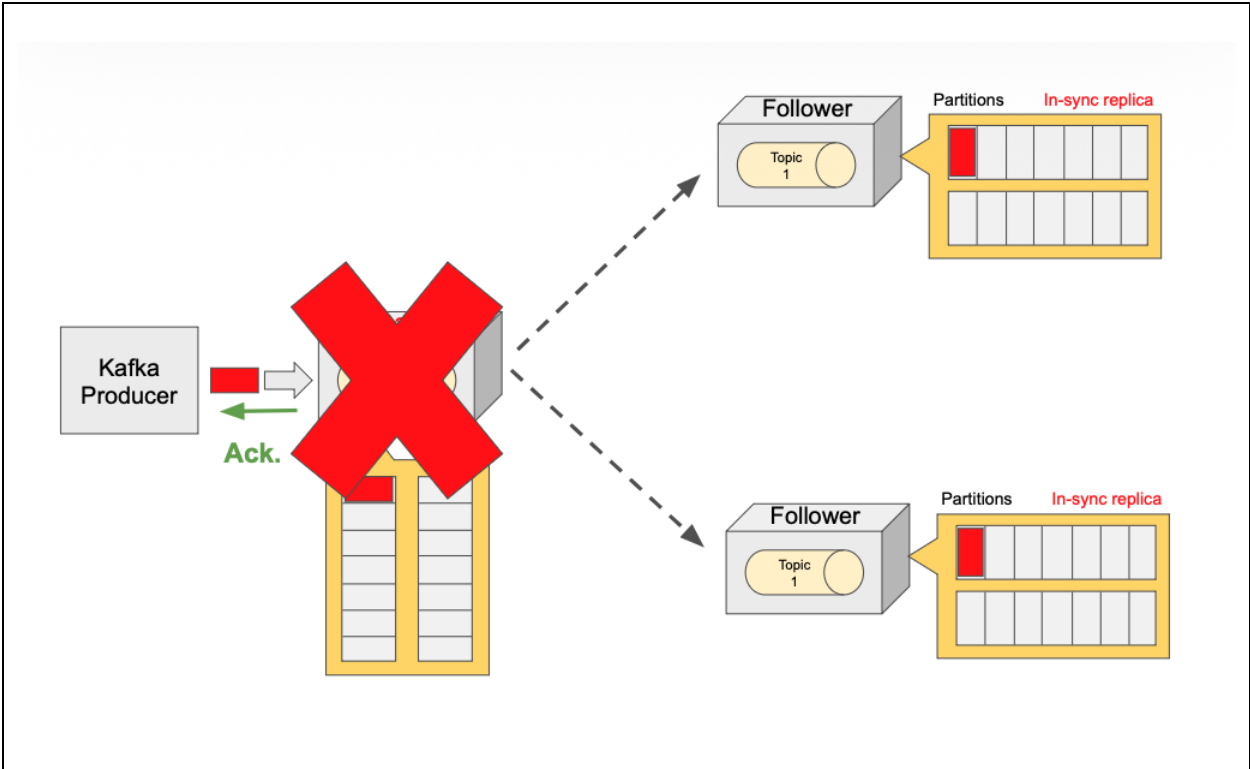
**Effect:**

- **Faster** than waiting for all replicas
- Still **reliable**, because at least the configured number of brokers have persisted the record

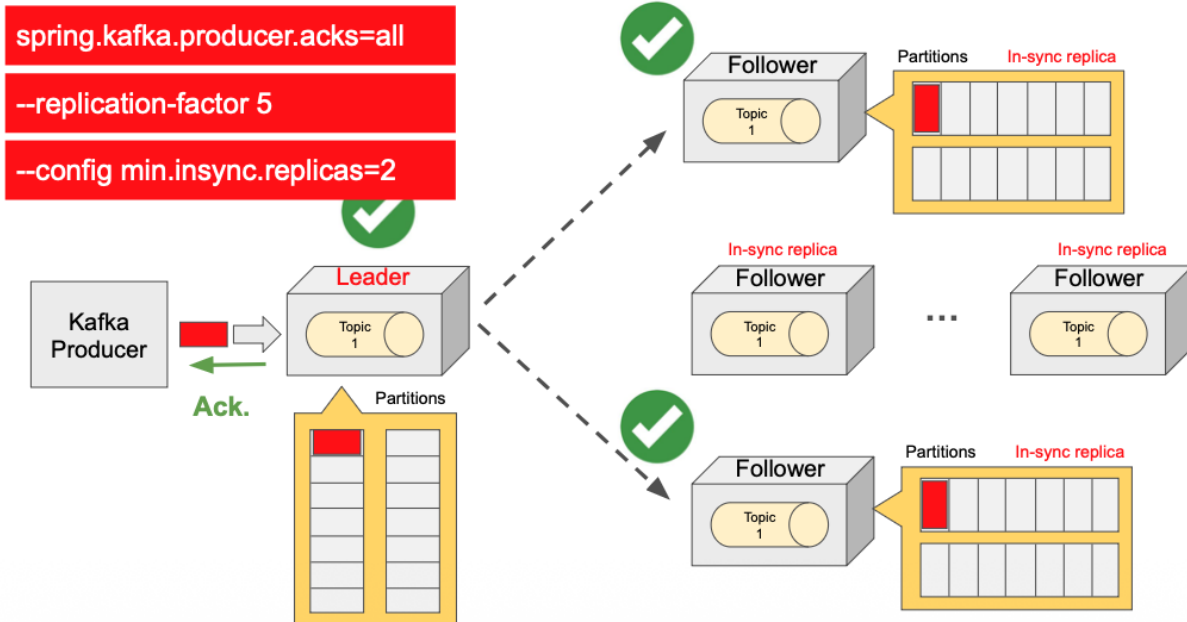
**Key takeaway**

Use producer acknowledgement settings (acks) **together with** replication and ISR-related settings (like min.insync.replicas) to tune the trade-off between:

- **Latency/throughput**
- **Durability and data-loss risk**



- **spring.kafka.producer.acks=all**  
Waits for an acknowledgement from all brokers.
- **spring.kafka.producer.acks=1**  
Waits for an acknowledgement from a leader broker only.
- **spring.kafka.producer.acks=0**  
Does not wait for an acknowledgement.



- **spring.kafka.producer.acks=all**

Waits for an acknowledgement from all brokers.

```
--replication-factor 5
```

```
--config min.insync.replicas=2
```

## Kafka Producer Retries?

**Context:** acks=all + min.insync.replicas

If you configure strong durability, for example:

- Producer acks=all (wait for all **in-sync replicas**)
- Broker/topic min.insync.replicas=3

...it means the broker will only accept the write if **at least 3 replicas are in-sync** (the leader + 2 followers). The producer will only get a successful acknowledgement once the leader and required in-sync replicas have persisted the record.

### What if a follower goes down?

If one follower goes down and the cluster no longer has enough in-sync replicas to satisfy min.insync.replicas=3, the write cannot be acknowledged successfully.

In that case, the producer typically receives a **retriable error** (temporary failure), and Kafka's default behavior is to **retry** sending until:

- the record is successfully delivered, or
- the **delivery timeout** is reached (default is **2 minutes**)

## Producer responses you may see

When a producer sends a record, one of the following may occur:

### 1) No response expected (acks=0)

If acks=0, the producer does not wait for acknowledgements. It sends and continues without knowing whether the broker stored the record.

- **Pros:** fastest
- **Cons:** no delivery guarantees

### 2) Success acknowledgement

With acks=1 or acks=all, the broker responds with a success acknowledgement if the write meets the acknowledgement requirements.

### 3) Error response

If the broker cannot complete the request, it responds with an error. Errors fall into two broad categories:

## ***Non-retriable (permanent) errors***

Retrying won't help. Example:

- **Record too large** (exceeds max.request.size / broker limits)

Result: producer fails the send and returns/raises an exception.

## ***Retriable (temporary) errors***

Retrying might succeed later. Examples:

- transient network issues
- leader temporarily unavailable (leader election)
- not enough in-sync replicas (e.g., ISR shrank below min.insync.replicas)

Result: producer automatically retries (you typically don't need custom logic).

## **Retry controls (key producer configs)**

Kafka automatically decides whether an error is retriable. You control *how long* / *how often* it retries using producer settings.

### **1) retries**

Controls **how many retry attempts** the producer will make.

- Default is very large (effectively “retry a lot”) but bounded by delivery.timeout.ms.
- If you set retries=10, the producer will try up to 10 retries (subject to timeout).

### **2) retry.backoff.ms**

Controls the **wait time between retries**.

- Default: 100ms
- Example: 1000ms means wait 1 second between retries.

## **Example behavior**

If:

- retries=10
- retry.backoff.ms=1000

Then on a retriable error the producer will retry **up to 10 times**, waiting **~1 second** between attempts (unless delivery.timeout.ms is hit first).

## Preferred control: delivery.timeout.ms

Kafka documentation generally recommends controlling retries via **total time spent trying**, rather than a fixed retry count.

### delivery.timeout.ms

Defines the **maximum total time** allowed for the entire send operation, including:

- time waiting to batch (linger.ms)
- time waiting for broker responses (request.timeout.ms)
- time spent retrying

Default: **120000 ms (2 minutes)**

If delivery.timeout.ms is exceeded, the producer fails the send with a timeout error.

## Related timing configs (must be consistent)

### linger.ms

Maximum time the producer waits to **batch** records before sending them.

- Default: 0 (send ASAP)
- Increasing linger.ms improves batching and can improve throughput (and compression effectiveness), at the cost of latency.

### request.timeout.ms

Maximum time to wait for a response from the broker for a **single request**.

- Default: 30000 ms (30 seconds)

## Important constraint

If you change delivery.timeout.ms, it must be  $\geq$  the effective time budget implied by:

- linger.ms
- request.timeout.ms (and retry behavior)

In practice, ensure delivery.timeout.ms is large enough that requests + retries can actually occur.

## Spring Boot / Spring Kafka configuration examples

### Example A: Reliable writes with bounded retries

properties

```
# Strong durability
spring.kafka.producer.acks=all
# Retry controls
spring.kafka.producer.properties.retries=10
spring.kafka.producer.properties.retry.backoff.ms=1000
# Total time budget for delivery (2 minutes here)
spring.kafka.producer.properties.delivery.timeout.ms=120000
# Single-request wait time (30s default)
spring.kafka.producer.properties.request.timeout.ms=30000
# Optional batching (0 default; increase for throughput)
spring.kafka.producer.properties.linger.ms=0
```

#### Example B: Prefer “time-based” delivery control (recommended pattern)

```
properties
spring.kafka.producer.acks=all
# Let retries be high, but cap total delivery time
spring.kafka.producer.properties.retries=2147483647
spring.kafka.producer.properties.delivery.timeout.ms=60000
spring.kafka.producer.properties.request.timeout.ms=30000
spring.kafka.producer.properties.retry.backoff.ms=250
```

This approach emphasizes: “Try as needed, but don’t exceed 60 seconds total.”

#### Example C: High-throughput, low criticality (acks=0)

```
properties
spring.kafka.producer.acks=0
spring.kafka.producer.properties.linger.ms=20
```

Fastest producer behavior, but you accept potential message loss.

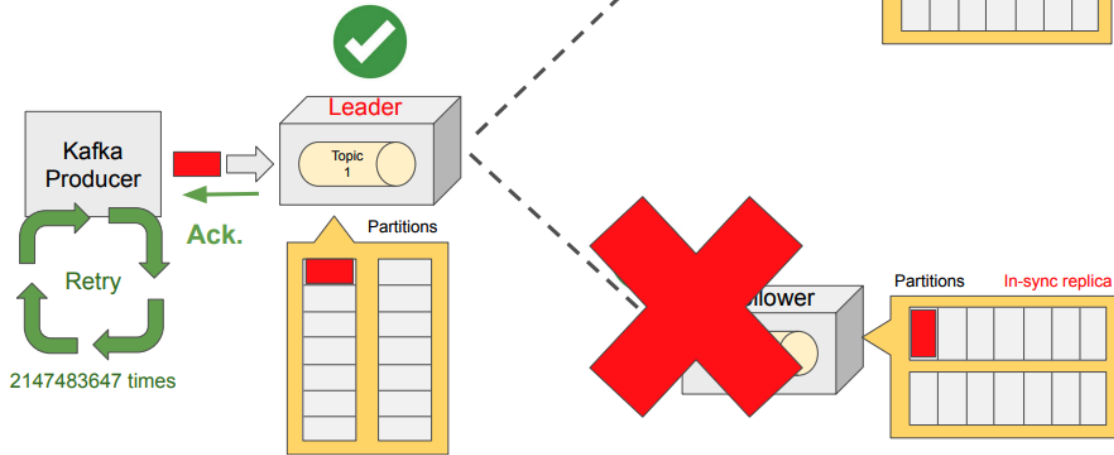
### Key takeaway

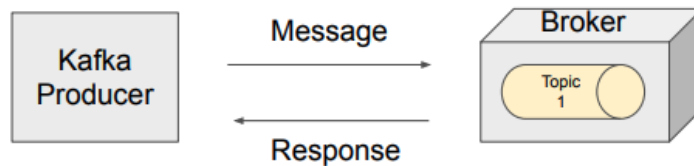
Use these properties together to define producer behavior under transient failures:

- **Durability requirement:** acks (often all for critical data)
- **Retry pacing:** retry.backoff.ms
- **Retry limit:** retries (optional; often less important than time-based control)
- **Overall time budget:** delivery.timeout.ms (most important)
- **Batching vs latency:** linger.ms
- **Per-request timeout:** request.timeout.ms

`spring.kafka.producer.acks=all`

`--config min.insync.replicas=3`





- **No response** - if the producer is configured with `acks=0`
- Acknowledgement (ACK) of Successful Storage
- **Non-Retryable Error** - A permanent problem that is unlikely to be resolved by retries.
- **Retryable Error** - A temporary problem that can be resolved by retrying the send operation.

`spring.kafka.producer.retries=10`

How many times Kafka Producer will try to send a message before marking it as failed. Default value is 2147483647.

`spring.kafka.producer.properties.retry.backoff.ms=1000`

How long the producer will wait before attempting to retry a failed request. Default value is 100 ms.

`spring.kafka.producer.properties.delivery.timeout.ms=120000`

The maximum time Producer can spend trying to deliver the message. Default value is 120000 ms (2 minutes).

`delivery.timeout.ms >= linger.ms + request.timeout.ms`

`spring.kafka.producer.properties.linger.ms=0`

The maximum time in milliseconds that the producer will wait and buffer data before sending a batch of messages. The default value is 0.

`spring.kafka.producer.properties.request.timeout.ms=30000`

The maximum time to wait for a response from the broker after sending a request. The default value is 30000 ms.

## Section 8: Idempotent Producer in Java

## Idempotent Kafka Producer Introduction?

---

### Why idempotence is needed

Kafka producers and brokers communicate over the network, so failures can happen even in the “happy path”.

#### Normal (happy path):

1. The producer sends a record to the broker.
2. The broker writes the record to the target topic/partition.
3. The broker sends an acknowledgement (ack) back to the producer.

#### Failure scenario (duplicates without idempotence):

1. The producer sends a record.
2. The broker successfully writes it.
3. The broker sends an ack, but a network issue prevents the ack from reaching the producer.
4. The producer assumes the send failed and retries (if retries are enabled).
5. The broker receives the same record again and writes it again.

Result: **duplicate records** appear in the topic.

### Why duplicates are a problem

In some systems duplicates are tolerable, but in others they can cause serious issues:

- **Banking:** a customer could be charged twice.
- **E-commerce:** an order could be shipped twice.

### What an idempotent producer does

An **idempotent producer** prevents duplicates caused by retries.

**Definition (practical):** An idempotent producer can resend the same record multiple times, but the broker will **store it only once** and **preserve ordering** (within the scope of Kafka’s idempotence guarantees).

### How it behaves with retries enabled

With idempotence enabled:

1. Producer sends a record.
2. Broker writes it and attempts to send an ack.
3. If the ack is lost and the producer retries:
  - a. The broker recognizes the retry as a duplicate send.

- b. The broker does **not** write the record again.
- c. The broker simply acknowledges it.

Result: **no duplicates** in the topic even when retries occur due to transient failures.

## How to enable idempotence

### Kafka producer property

The key setting is:

- `enable.idempotence=true`

### Spring Boot / Spring Kafka (application.properties)

properties

`spring.kafka.producer.properties.enable.idempotence=true`

### Native Kafka producer config (generic)

properties

`enable.idempotence=true`

### Important note about defaults

In newer Kafka versions, idempotence is commonly **enabled by default**.

Even so, it's recommended to **explicitly set** `enable.idempotence=true` to avoid accidental disabling due to conflicting configuration.

### Required supporting settings (to keep idempotence valid)

If you want idempotence to work reliably, these settings must not conflict:

#### 1) `acks=all`

The producer should wait for acknowledgements from all in-sync replicas.

properties

`acks=all`

#### 2) `retries > 0`

Retries must be enabled (a positive value). This is part of the safe delivery model for transient failures.

properties

`retries=10`

### 3) `max.in.flight.requests.per.connection <= 5`

This limits how many produce requests can be in-flight simultaneously without waiting for acknowledgements.

- Keeping it  $\leq 5$  helps the producer preserve ordering during retries/failures and avoid inconsistent ordering behavior.

properties

`max.in.flight.requests.per.connection=5`

### What happens if configs conflict?

- If idempotence is **not explicitly enabled**, conflicting settings may silently disable the default idempotence behavior.
- If you **explicitly set** `enable.idempotence=true` but configure incompatible values, Kafka will throw a **configuration exception** at startup (fail fast), which is usually what you want.

### Example: Recommended Spring Kafka configuration

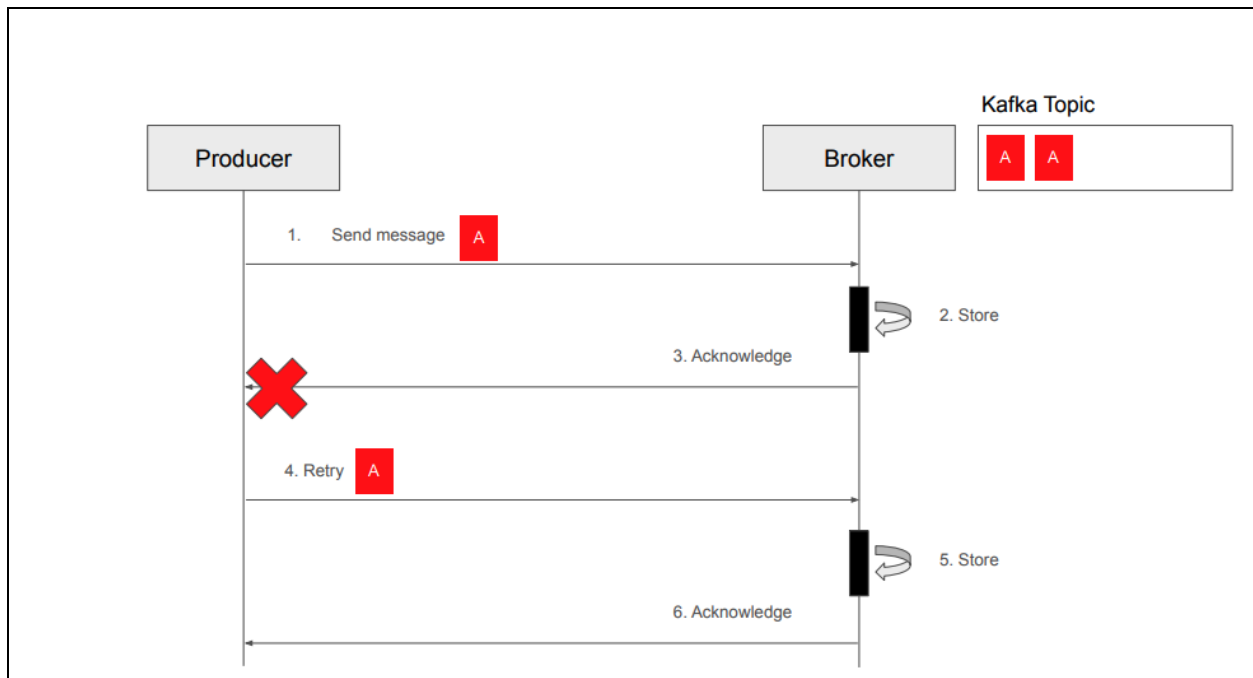
properties

`spring.kafka.producer.acks=all`

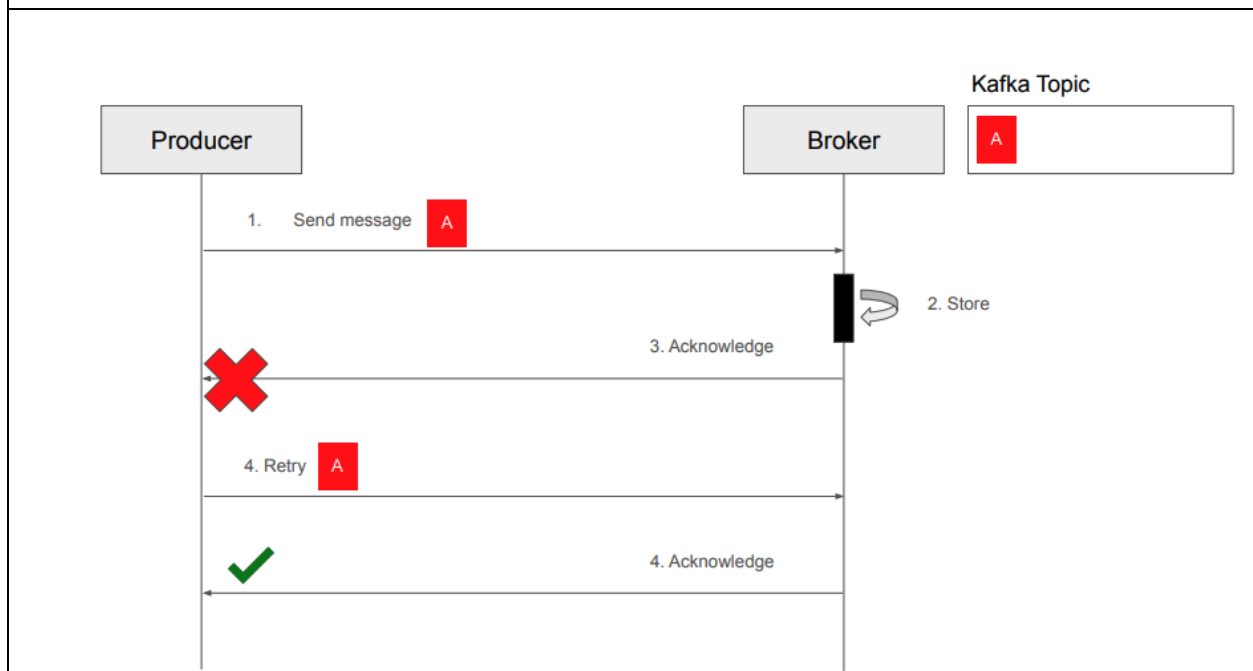
`spring.kafka.producer.properties.enable.idempotence=true`

`spring.kafka.producer.properties.retries=10`

`spring.kafka.producer.properties.max.in.flight.requests.per.connection=5`



An idempotent producer avoids duplicate messages in the log in the presence of failures and retries.



**enable.idempotence = true**

### **application.properties file**

```
spring.kafka.producer.properties.enable.idempotence=true
```

### **@Bean**

```
props.put (ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
```

**acks=all**

**retries=2147483647**

**max.in.flight.requests.per.connection=5**

### **application.properties file**

```
spring.kafka.producer.properties.acks=all  
spring.kafka.producer.properties.retries=2147483647  
spring.kafka.producer.properties.max.in.flight.requests.per.connection=5  
spring.kafka.producer.properties.enable.idempotence=true
```

**ConfigException**

## Section 9: Kafka Consumer – Spring Boot Microservice

## Introduction to Kafka Consumer?

---

**Products microservice** as a Kafka **producer**. It publishes events (messages) to a Kafka topic.

To read these events, you will create a new **Spring Boot microservice** that acts as a Kafka **consumer**. This consumer service will be called the **email notification microservice**, and it will continuously consume new messages from the Kafka topic.

### Kafka consumer does not delete messages

When a Kafka consumer reads a message from a topic, Kafka **does not delete** that message. The record remains in the topic and can still be read again (depending on consumer offsets and retention settings).

This also means you can run **multiple independent consumers**, and each consumer can receive its **own copy** of the messages (based on how they are grouped/configured).

### Topic partitions and message storage

Consider a topic named **Product Created Event Topic** with **3 partitions**.

- Messages in Kafka are stored inside **partitions**
- Each partition is an ordered log of records
- Each record has an **offset** (0, 1, 2, ...) within its partition

In the diagram representation:

- Partition 0 contains its own sequence of messages
- Partition 1 contains its own sequence of messages
- Partition 2 contains its own sequence of messages

### How consumers read from partitions

A consumer service polls Kafka for new records at a regular interval (this polling behavior is configurable via consumer properties).

### Parallel reads across partitions

Kafka reads from different partitions **in parallel**.

- **No ordering guarantee across partitions**
- **Ordering is guaranteed within a single partition**

Example (Partition 1):

- Offset 0 is always read before offset 1
- Offset 1 is always read before offset 2
- And so on

But Kafka does not guarantee that Partition 1 messages are processed before Partition 2 messages.

## Scaling consumers with partitions (consumer group concept)

If you run only **one consumer instance**, it can read from **all 3 partitions**.

If you run **three instances** of the same consumer microservice (as part of the same logical group), Kafka can assign:

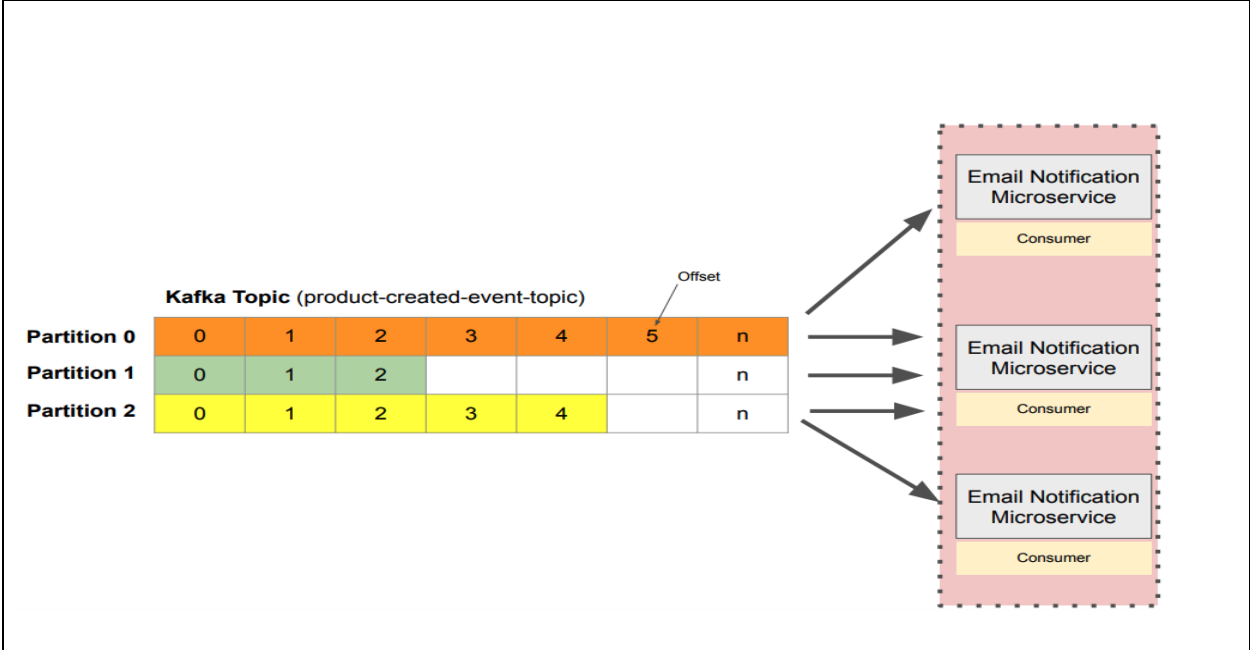
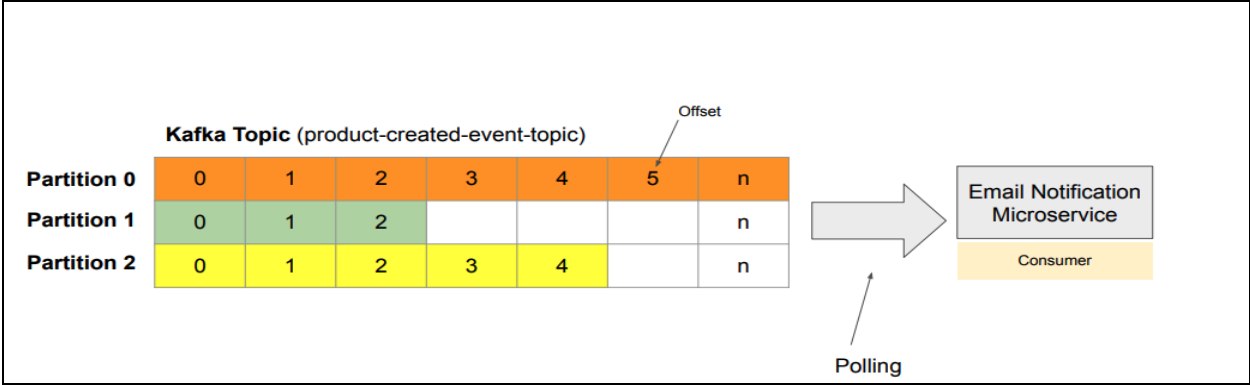
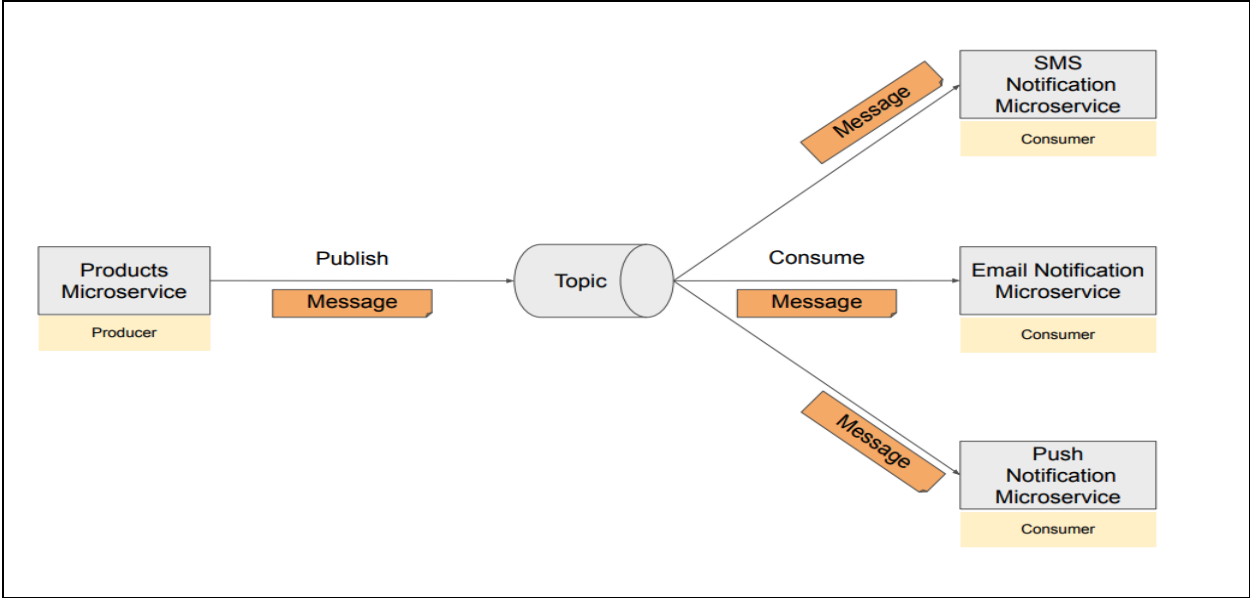
- Consumer 1 -> Partition 0
- Consumer 2 -> Partition 1
- Consumer 3 -> Partition 2

This lets you scale processing throughput because work is distributed across consumer instances. (You'll cover the detailed mechanics later in the lessons.)

## Message retention (how long messages stay)

Because Kafka doesn't delete records when consumed, messages remain in the topic until Kafka removes them based on **retention policy**.

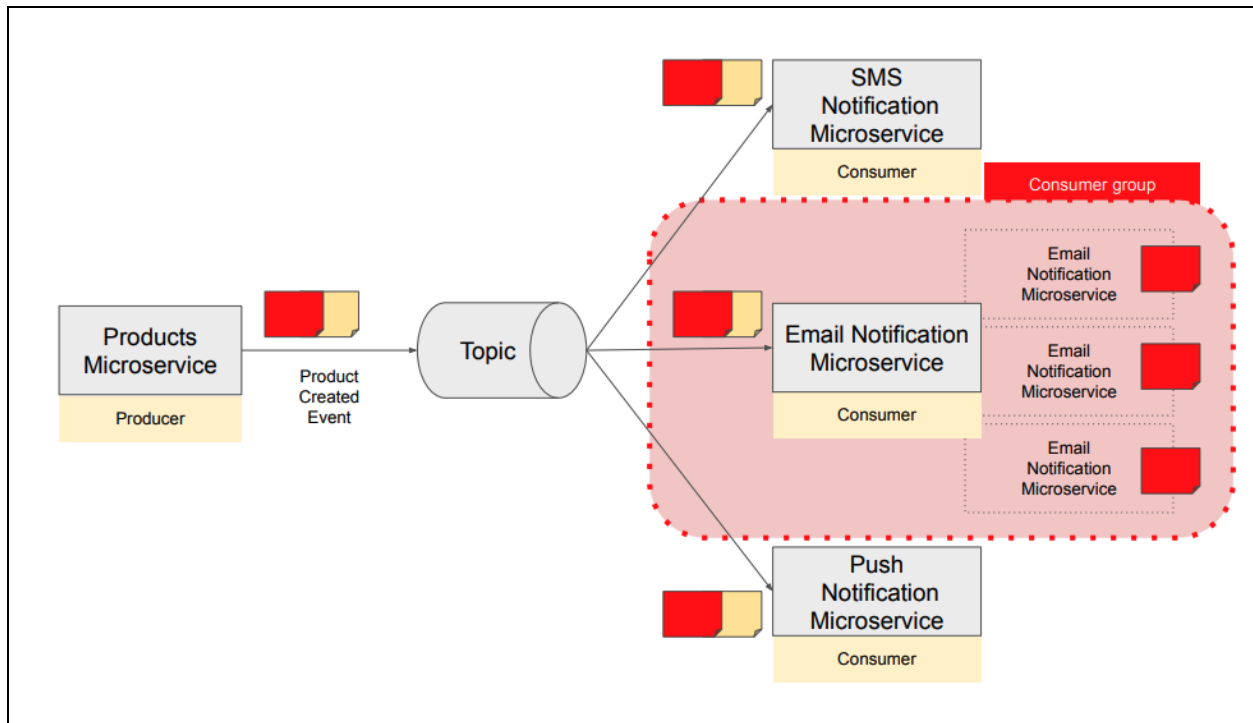
- Default retention is **168 hours (7 days)**
- This can be changed using Kafka topic configuration properties





## Section 13: Kafka Consumer – Multiple Consumers in a Consumer Group

## Introduction to Kafka Consumer Groups?



## Introduction to Kafka Consumer Groups — Documentation Version

### Scenario: one producer, multiple consumers

In this architecture:

- A **products microservice** acts as a **Kafka producer**.
- It publishes a **Product Created Event** to a **Kafka topic**.
- Multiple microservices listen to that topic as **Kafka consumers**.

When the producer publishes a new event, Kafka stores it in the topic. As soon as the event is available, consumers that subscribe to the topic can read it.

### Two different consumption patterns

#### 1) Different microservices: each one should receive a copy

If you have different consumer services such as:

- Email Notification Service
- SMS Notification Service
- Push Notification Service

...then **each service needs its own copy** of the Product Created Event.  
So all three services will consume the same event (each for its own purpose).

This is a common pattern where multiple independent services react to the same business event.

## The scaling problem: multiple instances of the same service

Now consider the **email notification microservice**.

If you scale it horizontally and run **4 instances** of the email service, you **do not** want all 4 instances to process the exact same message—because that would send 4 emails for the same event.

Instead, you want:

- The event to be processed **only once**
- By **any one** of the email-service instances

## What is a Kafka consumer group?

A **consumer group** is Kafka's mechanism for scaling a consumer application safely.

When multiple instances of the **same microservice** share the same group.id:

- Kafka treats them as members of **one consumer group**
- For each message/partition, Kafka ensures only **one group member** processes the data
- Work is distributed across the group members

This allows you to scale processing throughput without duplicating work.

## Why consumer groups help with performance

Consumer groups are especially useful when:

- The topic contains **lots of messages**
- One consumer instance cannot keep up
- You want faster processing by running **multiple instances** of the same consumer service

In this model:

- You can start multiple instances of the email notification microservice
- Group them into one consumer group
- Each instance processes a portion of the work

Even with multiple instances running, **each new message is processed once**, by exactly one instance in that consumer group.

### Kafka Consumer Rebalancing & Partition Assignment (Apache Kafka)

#### Scenario setup

Assume you have a Kafka topic named product-created-events with **3 partitions**. Messages published to this topic are distributed across these partitions.

You also have an **email-notification microservice** that acts as a **Kafka consumer** (part of a consumer group).

#### How partition assignment works

##### Single consumer instance

If you run **one** instance of the email-notification service in a consumer group, Kafka assigns that single consumer **all 3 partitions**.

That consumer reads from all partitions (in parallel) and processes messages.

##### Scaling out: adding more consumer instances

When load increases, you can scale by starting additional instances of the same consumer microservice (same group.id).

Example: start a **second** consumer instance.

- Kafka detects there are now **2 consumers** in the group.
- Kafka **reassigns** the partitions across the two consumers.
- After assignment:
  - One consumer might read **1 partition**
  - The other consumer might read **2 partitions**

Kafka tries to distribute partitions so work is shared.

#### What is rebalancing?

**Rebalancing** is the automatic process where Kafka **reassigns partitions** among consumers in the same consumer group.

Rebalancing happens whenever:

- A **new consumer joins** the group

- An **existing consumer** leaves the group (shutdown, crash, network issue)

### Important behavior during rebalancing

- Rebalancing is usually quick.
- But during rebalancing, consumers **pause consumption briefly** while assignments are updated.

After the rebalance completes, all active consumers resume consuming from their newly assigned partitions.

### Key rule: consumers cannot exceed partitions (within one group)

A critical constraint:

- **One partition can be consumed by only one consumer in a group at a time.**
- Therefore, you **cannot** have more active consumers than partitions for a given topic (within the same consumer group).

Examples:

- If a topic has **1 partition**, only **1 consumer** in the group can be active.
- If a topic has **3 partitions**, you can scale up to **3 consumer instances**.
- If you start a **4th consumer**, it will **sit idle** (no partition available to assign).

### Group coordinator and heartbeats

When a consumer joins a group, Kafka assigns one broker as the **group coordinator** for that consumer group.

Consumers regularly send **heartbeat** messages to the group coordinator to indicate they are alive and able to consume.

- Default heartbeat interval is about **3 seconds** (configurable).

### What happens when heartbeats stop?

If a consumer stops (crash, shutdown, network partition), it stops sending heartbeats.

When the broker detects missing heartbeats:

- It removes that consumer from the group
- It triggers a **rebalance**
- It reassigns the partitions among the remaining consumers

Kafka ensures again that:

- Each partition is assigned to **exactly one** consumer in the group.

## Summary

- **Partition assignment** determines which consumer reads which partitions.
- **Rebalancing** automatically redistributes partitions when consumers join/leave.
- **Max parallelism** in a consumer group is limited by the **number of topic partitions**.
- **Heartbeats** to the group coordinator control membership; missing heartbeats trigger rebalancing.

### Assigning a Microservice to a Kafka Consumer Group — Documentation Version

#### Goal

To scale a Kafka consumer microservice (like an email-notification service) without processing the same message multiple times, you assign all instances of that microservice to the same **consumer group** using `group.id`.

When multiple instances share the same `group.id`, Kafka ensures each message is processed **once** by **one** instance in the group.

### 3 ways to set `group.id` (Spring Kafka)

#### 1) Using `KafkaListener` annotation

You can set the consumer group directly on the listener:

```
java
@KafkaListener(topics = "product-created-events-topic", groupId = "product-created-
events")
public void handle(ProductCreatedEvent event) { ... }
```

##### Use when:

- You want the group assignment close to the listener logic
- You have different listeners that intentionally use different groups

#### 2) Using ConsumerFactory **configuration** (Java config)

You can set the group id in your Kafka consumer configuration class:

```
java
config.put(ConsumerConfig.GROUP_ID_CONFIG, "product-created-events");
```

Often this value is read from config using `Environment` (or

`Value)` rather than hardcoded.

##### Use when:

- You prefer central control of consumer settings
- You want one place to manage consumer configs (deserializers, retries, etc.)

### 3) Using application.properties / application.yml

Spring Boot supports configuring the group id via properties:

properties

`spring.kafka.consumer.group-id=product-created-events`

(or in YAML)

#### Use when:

- You want environment-based configuration (dev/test/prod)
- You want to avoid hardcoding group ids in code

### Recommended approach (best practice)

Pick **one primary approach** to avoid confusion. Most teams prefer:

- application.properties / application.yml for spring.kafka.consumer.group-id

...and keep Java config focused on advanced customization (deserializers, error handling, DLT, etc.).

### Using a custom property name (optional pattern)

You *can* define your own custom property key (for clarity), for example:

properties

`consumer.group.id=product-created-events`

Then read it inside ConsumerFactory:

java

```
config.put(  
    ConsumerConfig.GROUP_ID_CONFIG,  
    environment.getProperty("consumer.group.id")  
);
```

This makes it obvious the property is **custom**, not a Spring Boot standard key.

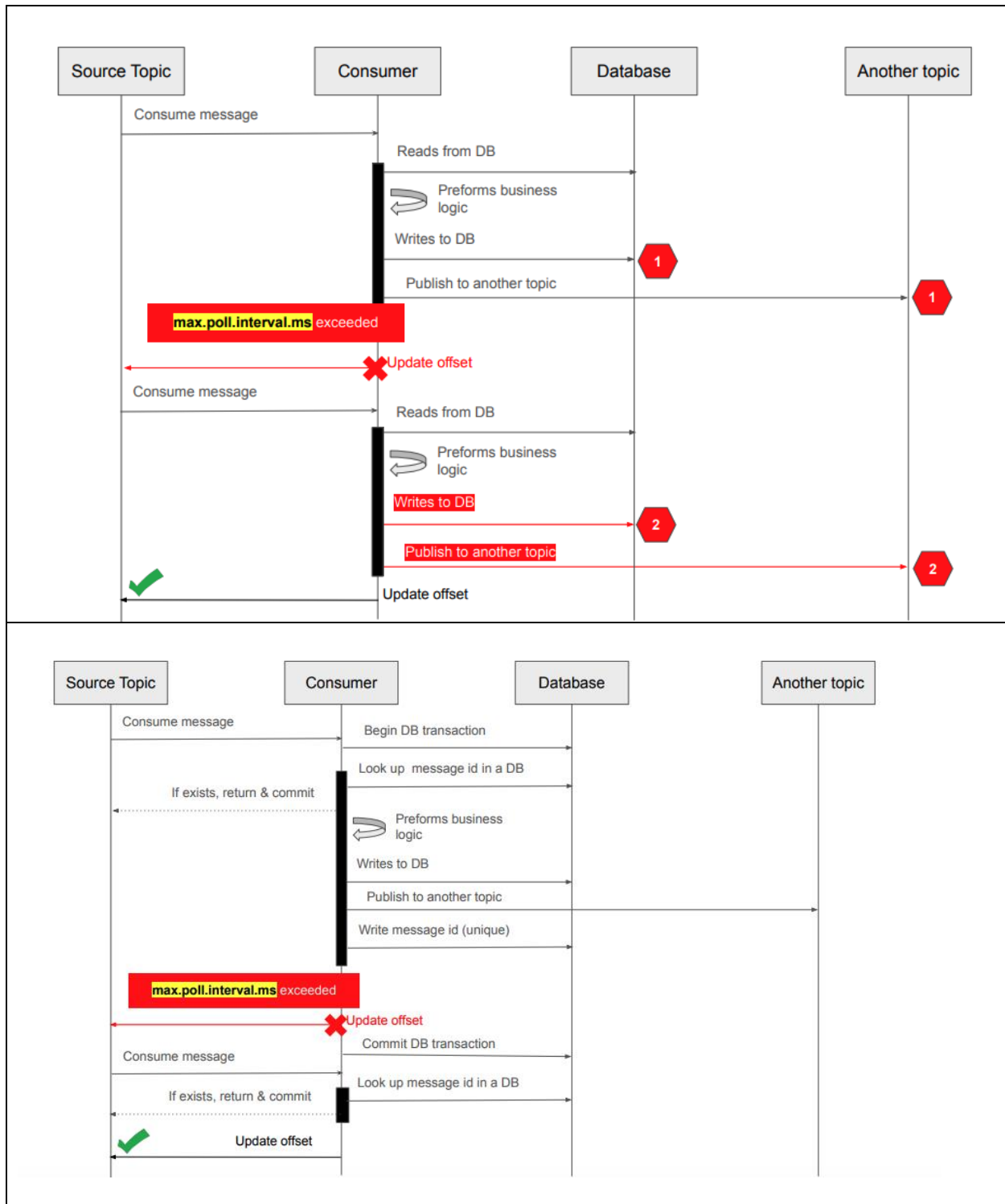
**Note:** If you use a custom property name, Spring Boot will not automatically apply it—you must read and apply it yourself (as shown above).

## Summary

- group.id determines which consumer group the microservice belongs to.
- You can set it via:
  - @KafkaListener(groupId=...)
  - ConsumerFactory Java configuration
  - spring.kafka.consumer.group-id in application config
- Prefer a **single consistent approach** across the project to keep configuration predictable.

## Section 14: Idempotent Consumer in Kafka?

## Idempotent Consumer – Introduction?



## Idempotent Kafka Consumer — Documentation Version

### What is an idempotent consumer?

An **idempotent consumer** is a consumer that can receive and process the **same message multiple times** without causing any side effects or data inconsistencies.

- Even if Kafka delivers a message again (due to retries, failures, or rebalancing),
- The consumer ensures the business outcome happens **only once**.

This is a key technique for building **effectively “exactly-once” processing behavior** at the application level.

### Why idempotent consumers are needed (duplicate processing scenarios)

#### Scenario A: Long processing time + max.poll.interval.ms exceeded

A common cause of duplicate consumption is when the consumer takes too long to process a record.

##### Flow:

1. Consumer polls messages from a topic and starts processing one message.
2. The message triggers long business logic:
  - a. read from database
  - b. compute
  - c. write to database
  - d. publish an event to another topic
3. Processing exceeds max.poll.interval.ms (maximum time allowed between polls).
4. Broker assumes the consumer is unhealthy, removes it from the group, and triggers a **rebalance**.
5. The message offset is **not committed**, so Kafka considers the record **not successfully consumed**.
6. Kafka delivers the same message again to another instance (or the same instance after rejoining).

**Risk:** the consumer may perform the business action twice:

- two database records written
- two downstream events produced
- potentially two payments made (critical issue)

#### Scenario B: Producer duplicates (non-idempotent producer)

If the producer is not idempotent, it may publish duplicate messages. Even if the consumer is stable, it can still receive duplicates and must handle them safely.

## Key idea: idempotence is only part of the solution

An idempotent consumer helps a lot, but by itself it does not eliminate every edge case. For high reliability, you typically combine:

- **Idempotent producer**
- **Idempotent consumer**
- **Transactions** (to group DB + Kafka operations so they succeed/fail together)

## Implementation approach: Deduplicate using a unique Message ID

### Core strategy

Every Kafka message should have a **unique ID** (e.g., in message headers).

When a consumer receives a message, it should:

1. Read the message ID.
2. Check a database table to see if that ID was processed before.
3. If already processed → **skip**
4. If new → process business logic, then record the message ID as processed.

## Recommended processing flow (with transactions)

### 1) Start transaction

Execute the consumer handler inside a database transaction (and optionally Kafka transaction depending on your design).

### 2) Check deduplication store first

- Look up message\_id in a database table (e.g., processed\_messages).

### 3) If already processed

- Return immediately.
- Commit transaction.
- Commit offset (so Kafka won't redeliver).

### 4) If not processed

- Execute business logic:
  - DB reads/writes
  - publish event to another topic (optional)
- Insert message\_id into processed\_messages
- Commit transaction

- Commit offset

### **Why this works during rebalancing / re-delivery**

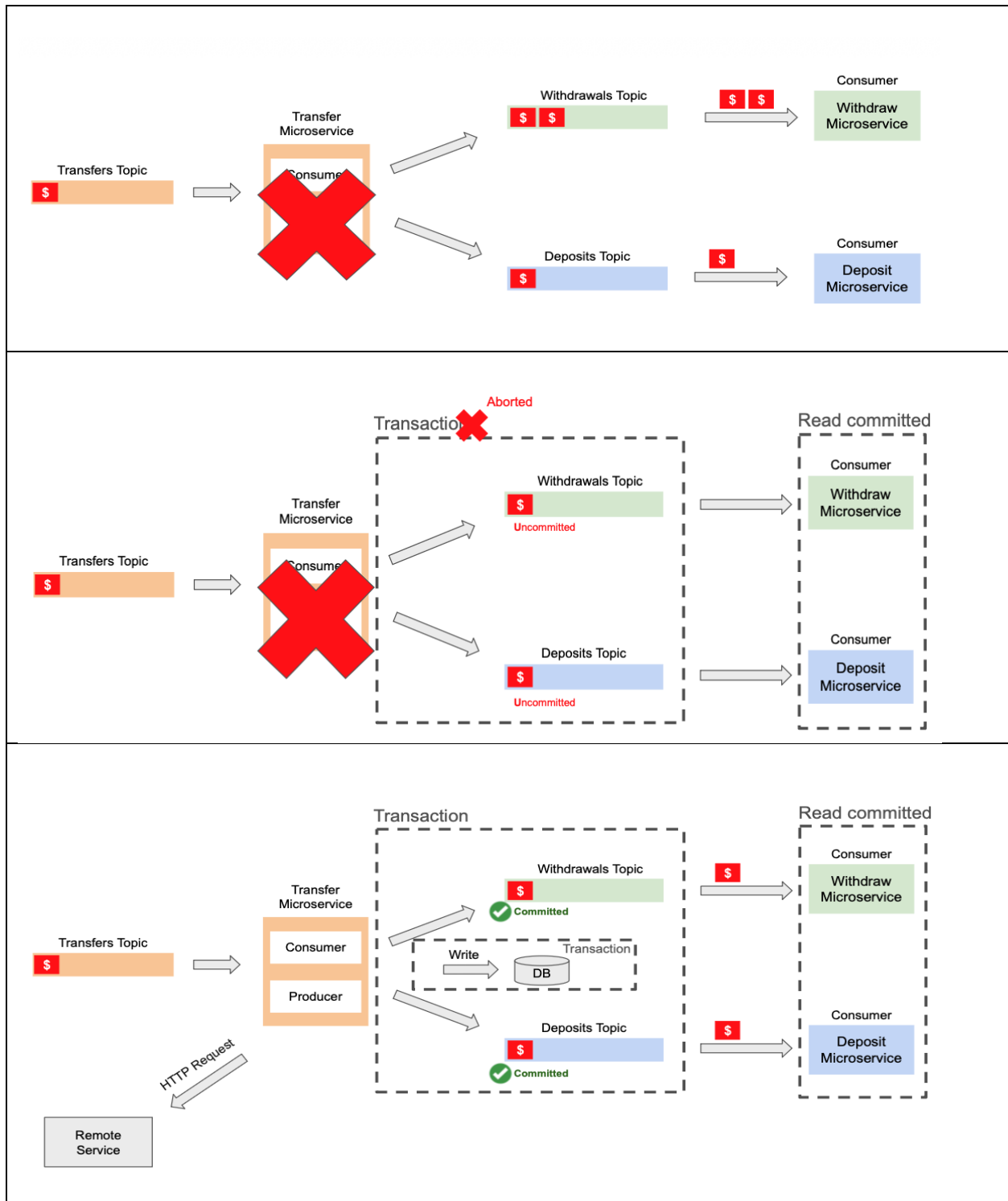
If the consumer is kicked out of the group due to `max.poll.interval.ms`, Kafka will redeliver the record.

But:

- Your original transaction might still have completed successfully (DB write already committed).
- So when the message comes again, the consumer checks the database, sees the message ID already processed, and skips it.
- The consumer then commits the offset so it won't be delivered again.

## Section 15: Apache Kafka Transactions

## Introduction to Transactions in Apache Kafka?



## Kafka Transactions — Documentation Version

### Two key goals of Kafka transactions

Kafka transactions are designed to support two important guarantees:

#### 1) All-or-nothing (atomic) behavior

Either **all Kafka operations inside a transaction succeed**, or **none of them take effect**.

- If any operation fails, the transaction is **aborted**
- Messages produced during the transaction are **not committed** and are not visible to consumers configured to read only committed data

#### 2) Exactly-once semantics (EOS)

Kafka aims for “delivered once and only once” behavior using a combination of:

- **Idempotent producer**: prevents duplicate sends caused by retries
- **Transactions**: prevent consumers from reading messages that are part of an incomplete transaction

### The common Kafka pattern: Consume → Process → Produce

A typical Kafka microservice often follows this workflow:

1. **Consume** a message from an input topic
2. **Process** it (business logic, DB interactions, validation, etc.)
3. **Produce** one or more messages to output topics for other services

### Example scenario (money transfer workflow)

Imagine a topic called transfer, consumed by a **Transfer microservice**.

When a new message arrives in the transfer topic, the Transfer microservice processes it and may produce events to downstream topics such as:

- withdrawal (handled by a Withdrawal microservice)
- deposits (handled by a Deposits microservice)

### What can go wrong without transactions?

Without transactions, this failure can occur:

1. Transfer microservice consumes a transfer request
2. It produces a message to the **withdrawal** topic
3. The service crashes before completing the rest of the workflow

4. Kafka re-delivers the original transfer message (since it was not fully “completed” from Kafka’s perspective)
5. The Transfer microservice processes it again and produces **another withdrawal message**
6. Later it produces **only one deposit message**

#### Result:

- Withdrawal happens **twice**
- Deposit happens **once**
- This is a serious inconsistency (especially for payment-like use cases)

### How transactions solve this problem

When transactions are enabled, the service performs multiple Kafka writes as a **single atomic unit**.

#### Transactional behavior (high-level)

1. Transfer microservice starts processing a message
2. It begins a Kafka transaction
3. It produces a message to withdrawal
4. It produces a message to deposits
5. If everything succeeds → it **commits** the transaction
6. If it crashes / an exception occurs → it **aborts** the transaction

#### Visibility rules: committed vs uncommitted

When a producer writes messages within a transaction:

- Those messages are initially marked as **uncommitted**
- Consumers configured with `isolation.level=read_committed` will **not see** them
- Only after the transaction commits do those messages become **visible**

So:

- If the service crashes mid-processing, the produced messages remain **uncommitted** and hidden
- Downstream services won’t process partial output

This prevents “withdraw twice, deposit once” outcomes caused by partial progress.

### What transactions guarantee (and what they don’t)

#### What Kafka transactions guarantee

- **Atomic writes to multiple Kafka topics**
- **All-or-nothing** publishing of a set of records
- Downstream consumers can be protected from partial results using `read_committed`

## What Kafka transactions do *not* automatically cover

Kafka transactions do not automatically include external side effects like:

- **HTTP requests** to other microservices
  - If an HTTP call succeeds and then the Kafka transaction aborts, the HTTP call is **not rolled back**
  - Handling this requires **compensating transactions** (undo logic across services)
- **Database transactions**
  - Kafka does not manage DB transactions by itself
  - However, frameworks like **Spring** can integrate Kafka + DB transactions so they can be coordinated (so DB changes and Kafka sends can be rolled back together depending on configuration)

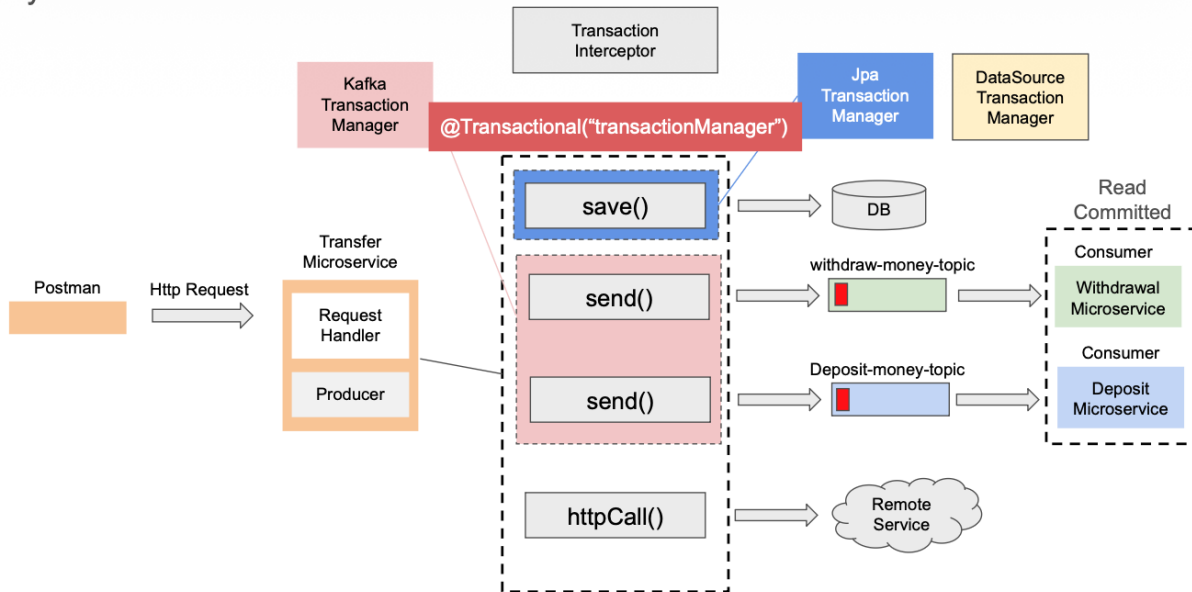
## Summary

- Kafka transactions enable **atomic, all-or-nothing** behavior for producing messages across one or more topics.
- Combined with an **idempotent producer** and consumers configured for `read_committed`, they help implement **exactly-once processing patterns**.
- Transactions solve partial publish problems but don't automatically roll back external calls (HTTP) and need integration for DB consistency.

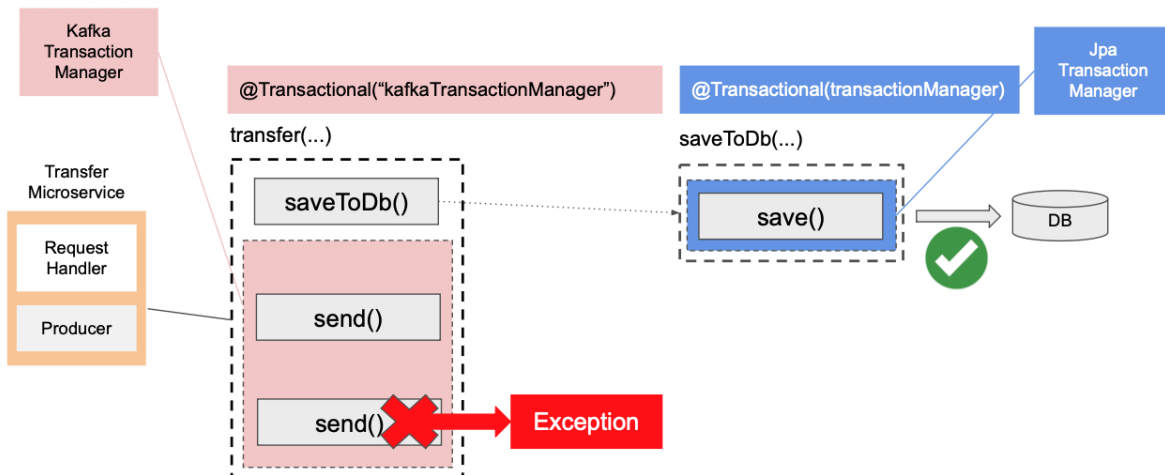
## Section 16: Apache Kafka and Database Transaction

## Kafka & Database Transactions – Introduction?

### Synchronized Transactions



### Nested Methods





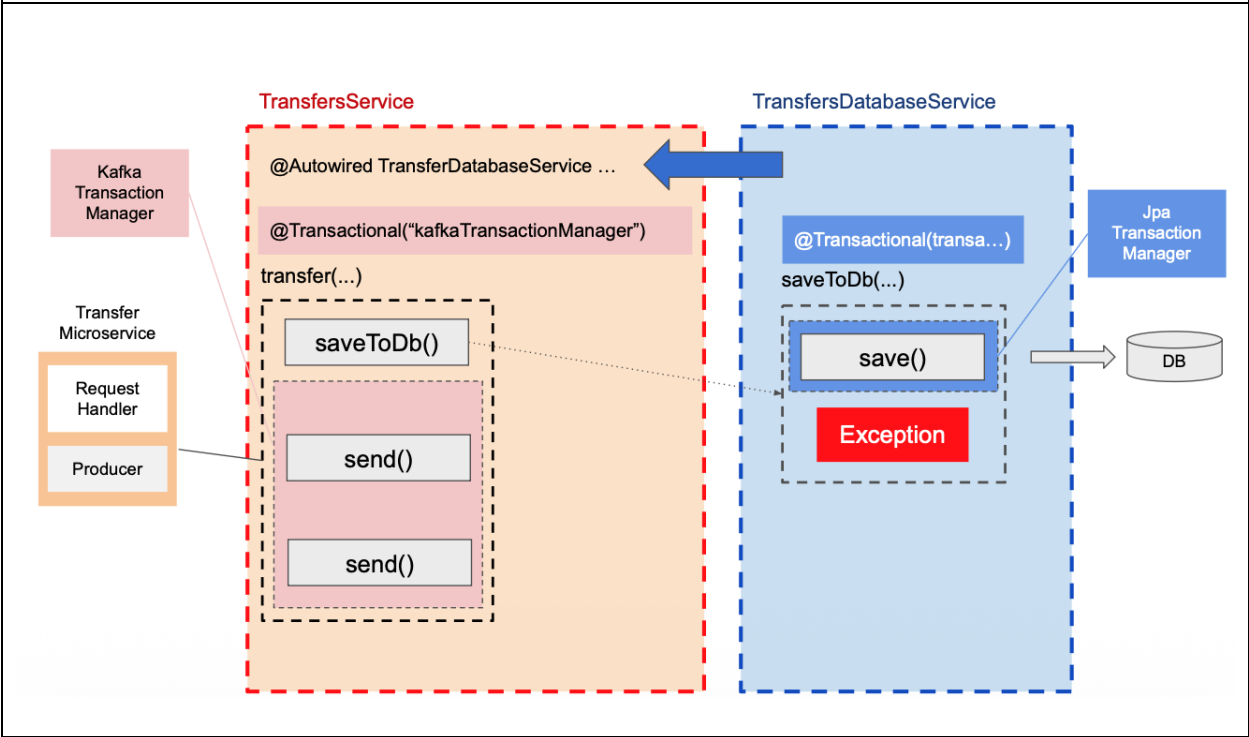
## Nested Methods

The diagram illustrates a nested transaction scenario. It shows a **Kafka Transaction Manager** (blue box) and a **Jpa Transaction Manager** (blue box). A **Transfer Microservice** (orange box) contains a **Request Handler** and a **Producer**. The **Request Handler** calls **transfer(...)**, which calls **saveToDb()**. The **Producer** calls **send()**. The **saveToDb()** method calls **save()**, which then interacts with the **DB** (Database). An **Exception** (red box) is shown occurring during the **save()** call, indicated by a red arrow pointing to the **save()** method call.

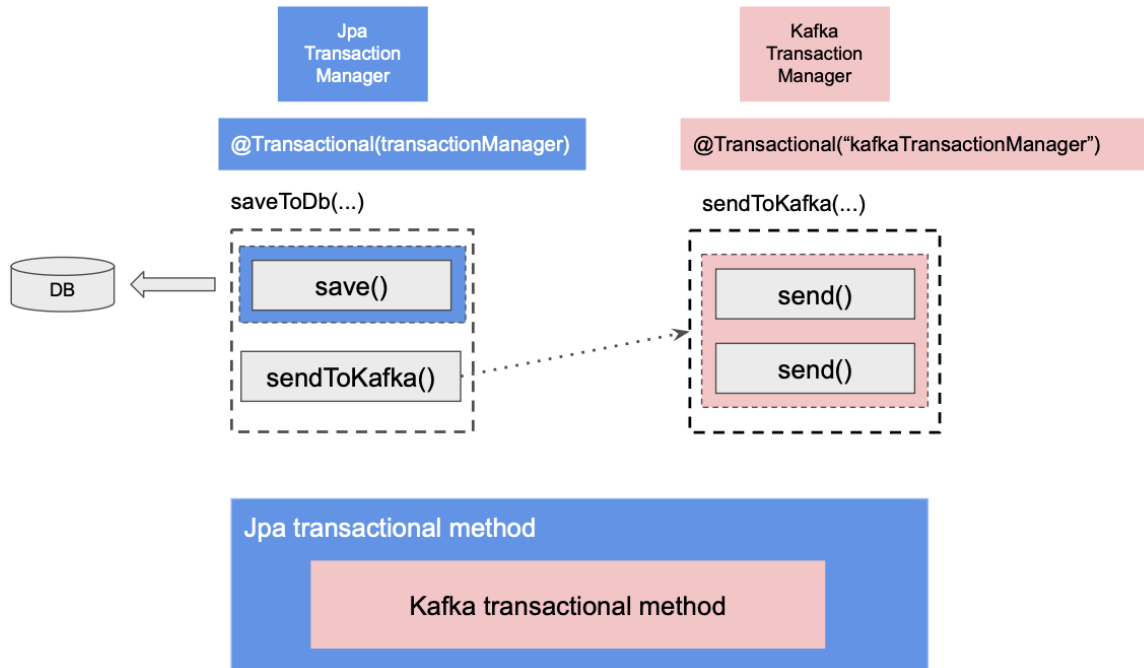
```
graph LR
    subgraph Kafka_TM [Kafka Transaction Manager]
        direction TB
        subgraph Transfer_MS [Transfer Microservice]
            direction TB
            subgraph Request_Handler [Request Handler]
                transfer(...)
            end
            subgraph Producer
                send()
            end
        end
    end

    subgraph Jpa_TM [Jpa Transaction Manager]
        direction TB
        subgraph saveToDb (...)
            save()
        end
    end

    transfer(...) -.-> saveToDb(...)
    saveToDb(...) --> DB[(DB)]
    send() --> Exception[Exception]
```



## Nested Methods



## Kafka + Database Transactions (Spring) — Documentation Version

### Goal

In many real systems, a single request triggers **both**:

- **Database work** (e.g., saving transfer state, audit logs)
- **Kafka work** (publishing events such as “withdraw requested” and “deposit requested”)

The goal is to ensure **consistent outcomes**:

- If something fails, you typically want **both Kafka sends and DB updates to roll back**
- If everything succeeds, you want **DB + Kafka** changes to be committed in a predictable order

## Reference flow: Transfer microservice

### Components

- **Transfer microservice**
  - Receives an HTTP request (Postman/cURL)
  - Runs the business logic

- Produces events to Kafka
- Persists transfer details into a database table (in this section)
- **Kafka topics**
  - withdraw-money-topic
  - deposit-money-topic
- **Consumer microservices**
  - Withdrawal microservice consumes withdraw topic
  - Deposit microservice consumes deposit topic
  - Both consumers are configured to read **committed only** (read\_committed) so they do not process uncommitted/aborted transactional records

## Kafka transactions (producer-side)

In the transfer service, Kafka transactions are enabled so that multiple sends are grouped into one atomic unit.

### What it means

When the method that sends messages is transactional:

- Sending to withdraw-money-topic
- Sending to deposit-money-topic

...both happen inside **one Kafka transaction**, typically managed by a **Kafka Transaction Manager**.

### Outcome

- If the Kafka transaction commits:
  - Both records become **committed** and visible to read\_committed consumers
- If the Kafka transaction aborts:
  - Records are not committed and should not be processed by read\_committed consumers

## Adding database operations to the same flow

In the next step, the same service method also performs database work (e.g., repository.save(...)).

Now the method performs **Kafka operations + DB operations** together.

This introduces a key design question:

**“Do Kafka operations and DB operations participate in one transaction or two?”**

That depends on **which transaction manager** starts the transaction for the method.

## Two transaction managers involved

In this section, you effectively have:

- **Kafka Transaction Manager**
  - Manages Kafka producer transactions
- **Database transaction manager** (e.g., JpaTransactionManager / DataSourceTransactionManager)
  - Manages database transactions

## How Spring ties this together (TransactionInterceptor)

Spring uses a component often referred to conceptually as the **transaction interceptor**:

- When a method annotated with Transactional is invoked, the interceptor runs
- It starts a transaction **before** the method execution
- It commits or rolls back **after** the method finishes (depending on success/failure)

## Why Kafka can join a DB transaction

When the interceptor starts a **database transaction**, the KafkaTemplate can **synchronize** its own transactional behavior with that ongoing transaction.

So even though Kafka and DB have different transaction managers, Spring can coordinate them within the same annotated method.

## Commit order (important detail)

When both Kafka sends and DB updates participate in the same Spring-managed transaction:

- **Database commits first**
- **Kafka commits second**

So the overall effect is:

1. DB write is finalized
2. Kafka records are committed (and become visible to consumers)

## Common pitfall: splitting logic across methods incorrectly

You may refactor like this:

- **Outer method**: Kafka sends
- **Inner method**: database save

## Scenario A — Outer method uses Kafka transaction manager

If the **outer method** is annotated with

Transactional using **KafkaTransactionManager**, and the inner DB save is executed via a JPA repository:

- Kafka transaction is managed by Kafka TM
- Repository save runs under a **separate DB transaction** (managed by JPA TM)

**Result: two separate transactions.**

### *Consequence*

If an exception happens in Kafka flow:

- Kafka transaction rolls back
- DB transaction may still **commit successfully**

In most business workflows, this is **not desired** (you don't want the DB showing "transfer saved" if the events were not successfully committed).

## Recommended approach: make the DB transaction the "outer" transaction

To ensure consistent rollback across Kafka + DB operations:

### Use [Transactional](#) with the database transaction manager on the outer method

When the outer method is driven by the DB transaction manager:

- DB transaction is started by Spring
- KafkaTemplate synchronizes Kafka operations with that transaction
- If an exception occurs, Spring rolls back the overall transaction, causing:
  - DB rollback
  - Kafka transaction abort

This is the strategy that gives the most "all-or-nothing" behavior for DB + Kafka within the same service.

## Key takeaways

- **Kafka transactions** guarantee atomicity across Kafka sends, but do not automatically include DB work unless coordinated.
- If Kafka-managed transaction is the outermost scope, DB operations may run in a separate transaction and still commit.
- To have Kafka + DB behave like a single unit, the safest pattern is:

- **Outer Transactional managed by DB transaction manager**
  - KafkaTemplate participates via transaction synchronization
- Commit order in coordinated flows is typically:
  - **DB commit first, then Kafka commit**



## Orchestration Based Saga?

---

### Orchestration-Based Saga (Formatted Transcription)

#### What it is

In an **orchestration-based saga**, one microservice (often the **Order** service) contains a **Saga class**. This Saga class acts as an **orchestrator/manager** that:

- **Consumes events**
- **Decides the next step**
- **Publishes commands** to other microservices
- Continues until the transaction is completed or rolled back

#### Happy Path (Success Flow)

##### Step 1: Create Order

- **Order Service** receives an HTTP request to create a new order.
- It creates the order in its database.
- It publishes an event: OrderCreatedEvent

##### Step 2: Reserve Product

- **Saga** consumes OrderCreatedEvent
- Saga publishes command: ReserveProductCommand
- **Product Service** consumes ReserveProductCommand
- Product Service reserves stock and publishes: ProductReservedEvent

##### Step 3: Process Payment

- **Saga** consumes ProductReservedEvent
- Saga publishes command: ProcessPaymentCommand
- **Payment Service** consumes ProcessPaymentCommand
- If payment is successful, it publishes: PaymentProcessedEvent (or PaymentProcessEvent, depending on naming)

##### Step 4: Continue to Shipment (Optional extension)

At this point the order can be marked approved and shipment can begin. Depending on your design:

- you can start a **new saga**, or
- continue with **more steps in the same saga**

Example continuation:

- Saga consumes PaymentProcessedEvent
- Saga publishes: CreateShipmentTicketCommand
- **Shipment Service** consumes it and publishes: ShipmentTicketCreatedEvent
- Saga consumes ShipmentTicketCreatedEvent
- Saga publishes: ApproveOrderCommand
- **Order Service** (handler class) consumes ApproveOrderCommand, updates DB to APPROVED
- Order Service publishes: OrderApprovedEvent

## Failure Path (Compensation / Rollback Flow)

### Scenario: Payment fails

Assume:

- Payment Service cannot verify payment details or charge the user.
- Since payment failed:
  - we **cannot ship**
  - we **cannot keep product reserved**
  - we must **rollback** previous changes

### Key idea: Compensations happen in reverse order

A compensation is an action that **undoes** a previous successful step.

### Compensation Flow Example

#### 1) Payment failure event is published

- Payment Service publishes an event like:
  - CardAuthorizationFailedEvent (your transcription says “CART Authorization Failed event”, likely intended “Card”)

#### 2) Cancel product reservation

- Saga consumes CardAuthorizationFailedEvent
- Saga publishes command: CancelProductReservationCommand
- Product Service consumes it
- Product Service marks product back to **AVAILABLE**
- Product Service publishes: ProductReservationCancelledEvent

#### 3) Reject the order

- Saga consumes ProductReservationCancelledEvent
- Saga publishes command: RejectOrderCommand
- Order Service consumes it
- Order status changes from **OPEN/CREATED** to **REJECTED**
- Order Service publishes: OrderRejectedEvent (or equivalent)

## Summary Rules to Remember

- **Saga orchestrates the workflow**
- Saga does not “do the work”; it:
  - publishes commands
  - consumes events
  - decides what’s next
- On failure, Saga triggers **compensating transactions**
- Compensations should generally be executed in **reverse order** of the successful steps

### Reserve Product in Stock (Formatted Transcription)

#### Overview

This flow starts when a client places an order and continues through the **product reservation** step, ending when the saga triggers the **payment** step.

#### Step-by-Step Flow

##### 1) Client places an order (Orders Service)

- A client application sends an **HTTP POST** request to create/place a new order.
- The request is handled by the **Orders Controller**.
- The controller passes the request data to an **Orders Service** class.
- The Orders Service saves the order details into the **orders database table**.

##### 2) Orders Service publishes OrderCreatedEvent

- After the order is successfully stored, the Orders Service publishes an event:
  - OrderCreatedEvent
- This event is sent to the **orders events topic**.

This is the point where the **saga flow begins**.

##### 3) Saga consumes OrderCreatedEvent and publishes ReserveProductCommand

- The **Orders Saga** class consumes OrderCreatedEvent.
- The saga publishes a command (commands are treated like “instructions” to other services):
  - ReserveProductCommand
- This command is published to the **products commands topic**.

##### 4) Product Service reserves stock and publishes ProductReservedEvent

- The **Products Service** is subscribed to the products commands topic.
- It consumes ReserveProductCommand.
- It reserves the product in stock (updates inventory/state).
- After reserving the product, it publishes:
  - ProductReservedEvent
- This event is published to the **products events topic**.

##### 5) Saga consumes ProductReservedEvent and moves to payment

- The **Orders Saga** consumes ProductReservedEvent.

- The saga begins the next step: processing payment.
- It publishes:
  - ProcessPaymentCommand

### Summary (One-line)

**HTTP POST** → **save order** → OrderCreatedEvent → **saga** → ReserveProductCommand → **product reserves stock** → ProductReservedEvent → **saga** → ProcessPaymentCommand.

## Section 20: Saga: Part 2: Compensating Transaction?

---

## Introduction to Compensating Transaction in Saga?

---

### Introduction to Compensating Transactions in Saga (Formatted Transcription)

#### What a Saga is

A **Saga** is an event-handling component that manages a **sequence of local transactions** across multiple microservices.

In earlier lessons, the saga handled a simple flow:

- it consumed an event
- then published a command
- which triggered the next local transaction in the saga

#### Why compensating transactions are needed

If **one of the local transactions fails**, the saga must run a series of **compensating transactions**.

#### Goal of compensating transactions

Compensating transactions **undo changes** made by the earlier (successful) steps in the saga.

Example:

- If ProcessPaymentCommand fails,
- the saga must undo the preceding modifying transaction:
  - ReserveProductCommand (because stock was reserved and must be released)

#### Core rule: compensations happen in reverse order

This is a key point in the Saga pattern:

- **Modifying transactions happen forward**
- **Compensating transactions happen backward (reverse order)**

So you undo:

- the **last** successful modification first,
- then move backward step-by-step until the **first** modification is undone.

## Important note: only compensate steps that changed state

You **do not** need compensating transactions for saga steps that did **not** modify your system.

Example:

- If a saga step is only a **read operation**, there is nothing to undo.

## Good practice: store history of events/status changes

It's a best practice to store a history of what happened during the saga.

In the example system (Orders, Products, Payments):

- each microservice has its **own database**
- in the Orders microservice, we also store an **Order History** table

## How the history table works

- The **Orders table** stores the current state (e.g., CREATED, APPROVED, REJECTED)
- The **Order History table** stores an append-only log of status changes with timestamps

When compensation happens:

- the existing row in the **Orders table** is updated to the new status
- but the **History table is NOT updated or modified**
- instead, a **new history row is inserted**

## Why this is useful

- **Auditing**
- **Debugging**
- **Compliance**
- Similar concept to **event sourcing**, where you keep all past events to reconstruct state at any time.

## Key takeaway

When executing compensating transactions:

- **do not delete**
- **do not edit past history records**
- **only append new history records**





- Section 1: Introduction to Apache Kafka
  - Introduction
  - What is Microservice
  - Microservice vs Monolithic application
  - Microservice communication
  - Event driven architecture with Apache Kafka
  - Apache Kafka for Microservices
  - Messages and Events in Apache Kafka
  - Kafka Topic and Partitions
  - Ordering of events in Apache Kafka
- Section 2: Apache Kafka Brokers
  - What is Apache Kafka Broker?
  - Apache Kafka Broker: Leader and Follower Roles. Leadership balance
  - Multiple Kafka broker: configuration files
  - Multiple Kafka Broker: Storage folders
  - Starting multiple Kafka broker with KRaft
  - Stopping Apache Kafka Broker
- Section 3: Kafka CLI: Topics
  - Introduction to Kafka Topic CLI
  - Creating a new Kafka Topic
  - List and describe Kafka Topic
  - Deleting Kafka Topic
- Section 4: Kafka CLI: Producers
  - Introduction to Kafka Producer CLI
  - Producing Kafka Message without a Key
  - Sending Kafka Message as a Key:Value Pair
- Section 5: Kafka CLI: Consumers
  - Introduction to Kafka Consumer CLI
  - Consuming messages from Kafka topic from the beginning
  - Consuming new Kafka messages only
  - Consuming Key value pair messages from Kafka topic
  - Consuming Kafka message in order
- Section 6: Kafka Producer – Spring boot Microservice
  - Introduction to Kafka Producer
  - Kafka Producer – Introduction to synchronous communication style
  - Kafka Producer – A use case for asynchronous communication style
  - Kafka Producer configuration properties/yaml

- Creating Kafka Topic
  - Creating an Event class
  - Kafka Producer: Send Message Asynchronously
  - Kafka Asynchronous Send: Message Synchronously
  - Kafka Producer: Handle Exception in Controller
  - Logging Record Metadata information
- Section 7: Kafka Producer: Acknowledgements & Retries
  - Kafka Producer Acknowledgement: Introduction
  - Kafka Producer Retries: Introduction
  - Configure Producer Acknowledgements in Spring Boot Microservices
  - The min.insync.replicas configuration
  - Kafka Producer Retries
  - Kafka Producer Delivery & Request Timeout
  - Kafka Producer Spring Boot Configuration
- Section 8: Idempotent Producer in Kafka
  - Introduction to Idempotent Kafka Producer
  - Enable Kafka Producer Idempotence in application.properties
  - Enable Kafka Producer Idempotence in Spring bean
- Section 9: Kafka Consumer – Spring Boot Microservice
  - Introduction to Kafka Consumer
  - Kafka Consumer Configuration Properties
  - Kafka Consumer with @KafkaEventListener and @KafkaHandler annotation
  - Kafka Consumer spring bean configuration
  - Kafka Listener Container Factory
- Section 10: Kafka Consumer – Handle Deserializer Errors
  - Introduction to Error Handling in Kafka Consumer
  - Causing the deserialization problem
  - Kafka Consumer ErrorHandlerDeserializer
- Section 11: Kafka Consumer – Deal Letter Topic (DLT)
  - Introduction to Dead Letter Topic (DLT) in Kafka
  - Handle errors: The DefaultErrorHandler and DeadLetterPublishingRecoverer classes
  - Create and Configure KafkaTemplate object
- Section 12: Kafka Consumer – Exceptions and Retries
  - Introduction to Exception Handling in Kafka consumer and retries
  - Creating retryable and not retryable exceptions in Kafka
  - Configure DefaultErrorHandler with a list of not retryable exceptions

- Register RetryableException and define wait time interval
  - Throwing a RetryableException
  - Overview of a destination microservice
- Section 13: Kafka Consumer: Multiple Consumers in a Consumer Group
  - Introduction to Kafka Consumer Group
  - Rebalancing and partition assignment in Kafka
  - Assigning Microservice to a consumer group in Kafka
  - Multiple consumers consuming messages from Kafka Topic

- Section 14: Idempotent Consumer in Kafka
  - Idempotent Consumer – Introduction
  - Include a unique id into message header
  - Reading a unique id from message header
  - Adding database-related dependencies
  - Configure database connection details
  - Creating JPA Entity
  - Create JPA Repository
  - Storing a unique message id in a database table
  - Check if Kafka message was processed earlier
- Section 15. Apache Kafka Transactions
  - Introduction to Transactions in Kafka
  - Enable Kafka Transactions in application properties
  - Enable Kafka Transaction in the @Bean method
  - Kafka and @Transactional annotation
  - Rollback transaction for specific exception
  - Reading committed messages in Kafka Consumer
  - Kafka local transaction with KafkaTemplate
- Section 16: Kafka and Database Transactions
  - Kafka & Database Transaction – Introduction
  - Creating Jpa Transaction Manager
  - Synchronized Transaction: Saving to Database
  - Enable logging for Kafka and Jpa Transaction Manager
- Section 17: Integration Testing – Kafka Producer
  - Test class Annotation
  - Test method structure – Arrange, Act & Assert
  - Implementing the Arrange and Act Sections
  - Kafka Consumer Configuration in Test class
  - The setUp() and tearDown() methods
  - Verify the Kafka Producer Configuration Properties
  - Test Method for Idempotent Kafka Producer
- Section 18: Saga design pattern with Apache Kafka
  - Choreography Based Saga
  - Orchestration Based Saga
  - Reserve product in Stock – Introduction
  - Publish OrderCreatedEvent
  - Handle OrderCreatedEvent

- Send ReserveProductCommand
- Save order status in History database table
- Handle ReserveProductCommand
- Reserve Product in Stock
- Publish the ProductReserveEvent
- Publish ProductReservationFailedEvent
- Handle the ProductReserveEvent
- Handle the ProcessPaymentCommand
- Handle Process Payment Command
- Publish Payment Processed and Payment Failed
- Handle PaymentProcessedEvent
- Handle the ApproveOrderCommand
- Handle the OrderApprovedEvent
- Section 19: Saga Part 2: Compensating Transaction
  - Introduction to Compensating Transaction in Saga
  - Handle PaymentFailedEvent
  - Publish CancelProductReservationCommand
  - Handle PaymentFailedEvent
  - Publish CancelProductReservation Command
  - Handle CancelProductReservationCommand
  - Publish ProductReservationCancelled Event
  - Handle ProductReservationCancelledEvent
  - Handle the RejectOrderCommand
- Section 20: Kafka Connect
  - Introduction to Kafka Connect
  - When and why to use Kafka Connect?
  - Kafka Connect Core Components: workers, connectors, and Tasks
  - Kafka Connect Key Components: Transforms, Converters